

Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών

Memory Consistency & Synchronization

Νεκτάριος Κοζύρης, Διονύσης Πνευματικάτος & Κωστής Νίκας
{nkoziris, pnevmati, knikas}@cslab.ece.ntua.gr

8ο εξάμηνο ΣΗΜΜΥ – Ακαδημαϊκό Έτος: 2025-26

<http://www.cslab.ece.ntua.gr/courses/advcomparch/>

Figures are from:

- “**Parallel Computer Architecture: A Hardware/Software Approach**”, D. E. Culler, J. P. Singh, Morgan Kaufmann Publishers, 1999.
- “**A Primer on Memory Consistency and Cache Coherence**” - 2nd edition, Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. Morgan & Claypool Publishers 2020 (Springer Open Access).
- Slides from cis601-cis.upenn.edu (Joe Devietti/Spring2016).

Πηγές/Βιβλιογραφία

- **“Parallel Computer Architecture: A Hardware/Software Approach”**, D. E. Culler, J. P. Singh, Morgan Kaufmann Publishers, 1999.
- **“Transactional Memory”**, D. Wood, Lecture Notes in ACACES 2009.
- **“A Primer on Memory Consistency and Cache Coherence”**, 2nd edition, Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood (Springer Nature Open Access - <https://library.oapen.org/handle/20.500.12657/61248>).
Also: https://pages.cs.wisc.edu/~markhill/papers/primer2020_2nd_edition.pdf
- **“Cache Coherence”**, Onur Mutlu, Computer Architecture 2015 course, Lectures 28 & 29 slides – Carnegie Mellon University.
 - <http://www.ece.cmu.edu/~ece447/s15/lib/exe/fetch.php?media=onur-447-spring15-lecture28-memory-consistency-and-cache-coherence-afterlecture.pdf>
 - <https://www.youtube.com/watch?v=JfjT1a0vi4E&t=4106s>
 - <http://www.ece.cmu.edu/~ece447/s15/lib/exe/fetch.php?media=onur-447-spring15-lecture29-cache-coherence-afterlecture.pdf>
 - <https://www.youtube.com/watch?v=X6DZchnMYcw>
- **“Computer Architecture: A Quantitative Approach”**, 6th edition, John Hennessy and David Patterson, Morgan Kaufmann (Elsevier), 2019.
- SC-TSO slides from cis601-cis.upenn.edu <https://www.cis.upenn.edu/~deviitti/classes/cis601-spring2016/>

Παράλληλες Αρχιτεκτονικές και Εφαρμογές

- Παράλληλες αρχιτεκτονικές (CMPs, SMPs, SMTs κτλ)
- Ιδανικές για παράλληλες εφαρμογές
 - Η πλειοψηφία των εφαρμογών δημιουργεί πολλαπλά threads που δουλεύουν κάτω από κοινή μνήμη
- Parallel Computer Architecture
 - Σχεδιασμός συστημάτων που επιτυγχάνουν υψηλές επιδόσεις ξεπερνώντας τόσο τα σειριακά όσο και τα παράλληλα bottlenecks
 - Ο προγραμματιστής πρέπει να μπορεί να γράφει σωστά παράλληλα προγράμματα που επιτυγχάνουν υψηλή επίδοση
- Κίνδυνοι με κοινή μνήμη σε παράλληλες αρχιτεκτονικές:
 - Ύπαρξη πολλαπλών αντιγράφων σε διαφορετικά επίπεδα της ιεραρχίας μνήμης → Coherence protocols = SOLVED!
 - Ανάγκη συγχρονισμού → Μεταφορά μοντέλων από uniprocessor systems

Ordering of Operations

- Operations: A, B, C, D
 - Με ποια σειρά πρέπει να εκτελέσει το hardware τις λειτουργίες αυτές;
- Ορίζεται από την ISA
 - “Συμφωνία” μεταξύ προγραμματιστή και αρχιτέκτονα.
- Διατήρηση μιας συγκεκριμένης σειράς βοηθά τον προγραμματιστή
 - Εύκολο debugging
 - State recovery, exception handling
- Διατήρηση μιας συγκεκριμένης σειράς δυσκολεύει τον αρχιτέκτονα
 - Αύξηση πολυπλοκότητας για την επίτευξη υψηλών επιδόσεων

Memory Ordering – Single Processor

- von Neumann Model
- Sequential order
 - Τα loads & stores εκτελούνται με τη σειρά που ορίζονται στο πρόγραμμα
- Out-of-order execution?
 - Οι εντολές γίνονται commit/retire με τη σειρά που εμφανίζονται στο πρόγραμμα
- Πλεονεκτήματα
 - Precise architectural state (in order!)
 - Consistent architectural state για διαφορετικές εκτελέσεις του προγράμματος → Debugging!
- Μειονεκτήματα
 - Hardware overheads, complexity
 - Μειωμένη επίδοση και κλιμακωσιμότητα

Memory Ordering – Multiprocessors

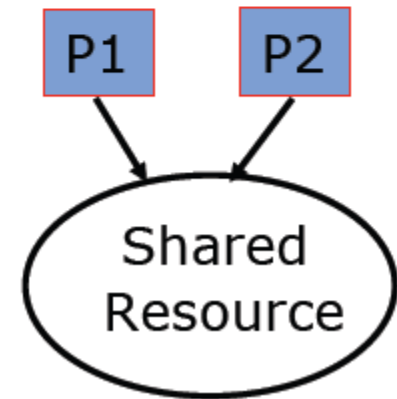
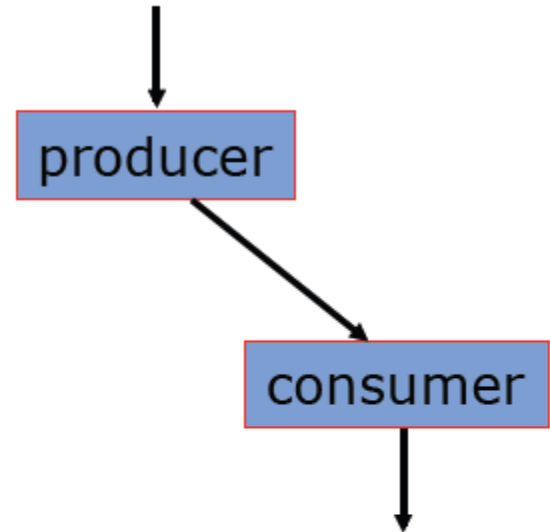
- Κάθε επεξεργαστής/thread υλοποιεί το μοντέλο von Neumann
 - Οι λειτουργίες κάθε νήματος εκτελούνται με τη σειρά που ορίζεται από το πρόγραμμα
- Πολλαπλοί επεξεργαστές εκτελούν TAYTOXPONA memory operations
- Με ποια σειρά βλέπει η μνήμη τα memory operations από όλους τους επεξεργαστές;
 - Είναι η σειρά/διάταξη των operations ίδια για όλους τους επεξεργαστές;

Memory Ordering – Multiprocessors

- Verification
 - Υπάρχει σειρά που θα τύχει κάποτε στην οποία το πρόγραμμα αποτυγχάνει;
- Debugging
 - Θα θέλαμε να έχουμε την ίδια σειρά σε κάθε εκτέλεση του προγράμματος → Repeatability
- Ορθότητα
 - Η σειρά εκτέλεσης των memory operations μπορεί να οδηγήσει σε λάθος εκτέλεση όταν αφορούν κοινά/διαμοιραζόμενα δεδομένα
 - Συγχρονισμός: Οι παράλληλες αρχιτεκτονικές πρέπει να διασφαλίζουν τη σωστή εκτέλεση των synchronization primitives προκειμένου οι προγραμματιστές να μπορούν να χρησιμοποιήσουν σωστά διαμοιραζόμενα δεδομένα
- Επίδοση και overheads

Συγχρονισμός

- Η ανάγκη για συγχρονισμό προκύπτει όποτε υπάρχουν ταυτόχρονες διεργασίες σε ένα σύστημα (ακόμα και σε *uniprocessor* σύστημα)
- **Μοντέλο παραγωγού – καταναλωτή:** ο καταναλωτής θα πρέπει να περιμένει μέχρι ο παραγωγός να παράξει δεδομένα
- **Αμοιβαίος αποκλεισμός:** εξασφαλίζει ότι μία μόνο διεργασία μπορεί να χρησιμοποιήσει έναν κοινό πόρο σε μια δεδομένη στιγμή



Παράδειγμα

`A = flag = 0;`

Processor 0

```
A = 1;  
flag = 1;
```

Processor 1

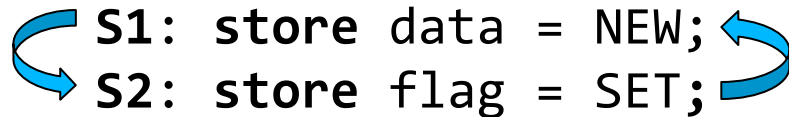
```
while (!flag); //spin  
print A;
```

- Τι μας λέει η *διαίσθηση*: ο P1 τυπώνει A=1
- Τι μας εγγυάται η *συνάφεια* μνήμης: απολύτως τίποτα!
 - απλά μας εξασφαλίζει ότι η καινούρια τιμή του A *κάποια στιγμή* θα γίνει ορατή από τον P1
 - ο P1 μπορεί να δει την εγγραφή του flag *πριν* την εγγραφή στο A !
Πώς;
 - τα μηνύματα του coherence protocol για την εγγραφή του A μπορεί να καθυστερήσουν κάπου στο δίκτυο διασύνδεσης
 - ο write buffer του P0 μπορεί να αναδιατάσσει τις εγγραφές
- Σε πραγματικά συστήματα, ο παραπάνω κώδικας μπορεί να «δουλεύει» μερικές φορές, ενώ άλλες όχι...

Producer-Consumer Example

// Initial shared: **data** = 0, **flag** = 0

Core C1:

 **S1: store data = NEW;**
S2: store flag = SET;

Core C2:

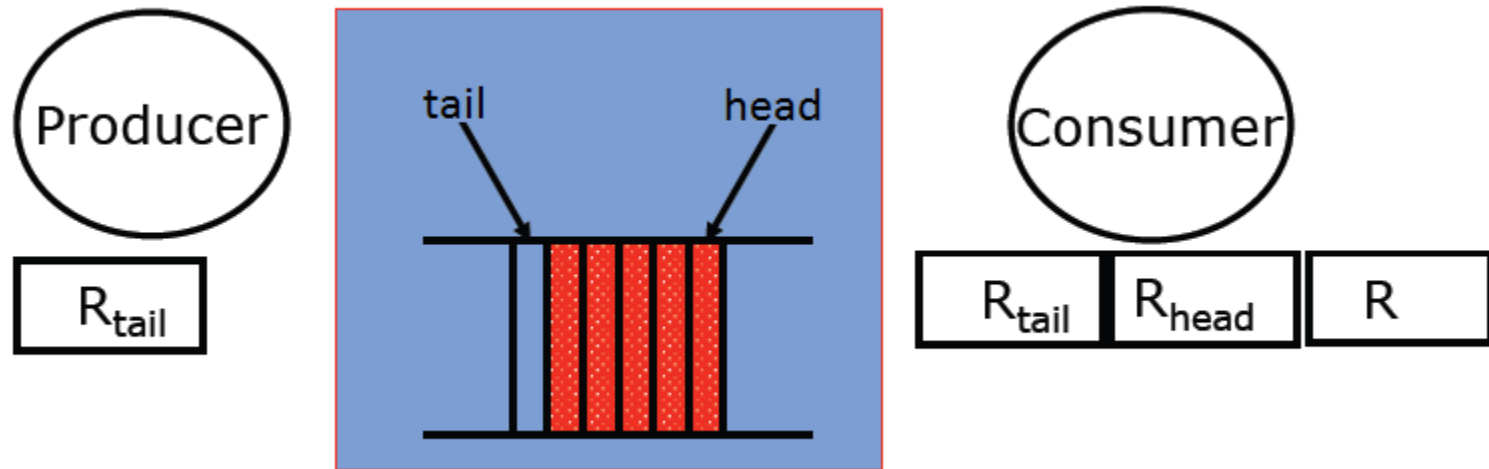
L1: load r1 = flag;
B1: if (r1 != SET) goto L1;
L2: load r2 = data;

Will **r2** always be set to **NEW**?

No, S1 and S2 may get reordered

The memory system or compiler may reorder S1 and S2. If **flag** is **set** *before data is committed*, Core C2 may read the stale value of data.

Producer-Consumer (same as before)



Producer posting Item x:

```
Load  $R_{tail}, (tail)$   
Store ( $R_{tail}$ ), x  
 $R_{tail} = R_{tail} + 1$   
Store (tail),  $R_{tail}$ 
```

Το πρόγραμμα είναι γραμμένο με την υπόθεση ότι οι εντολές εκτελούνται σε σειρά.

Consumer:

```
Load  $R_{head}, (head)$   
spin: Load  $R_{tail}, (tail)$   
if  $R_{head} == R_{tail}$  goto spin  
Load  $R, (R_{head})$   
 $R_{head} = R_{head} + 1$   
Store (head),  $R_{head}$   
consume( $R$ )
```

Producer-Consumer (same as before)

Producer posting Item x:

```
Load Rtail, (tail)
1 Store (Rtail), x
  Rtail = Rtail + 1
2 Store (tail), Rtail
```

Ο tail pointer μπορεί να ανανεωθεί πριν την εγγραφή του x!

Consumer:

```
Load Rhead, (head)
spin: Load Rtail, (tail) 3
      if Rhead == Rtail goto spin
      Load R, (Rhead) 4
      Rhead = Rhead + 1
      Store (head), Rhead
      consume(R)
```

- Ο προγραμματιστής υποθέτει ότι αν η 3 πραγματοποιηθεί μετά τη 2, τότε η 4 πραγματοποιείται μετά την 1.
- Προβληματικές ακολουθίες:
 - 2, 3, 4, 1
 - 4, 1, 2, 3

- Χρειαζόμαστε ένα μοντέλο σειριοποίησης για λειτουργίες μνήμης :
 - Σε ίδιες ή διαφορετικές θέσεις μνήμης
 - Από ένα ή πολλαπλά threads/processes

➤ Μοντέλο συνέπειας μνήμης

Memory Consistency

- **Συνάφεια μνήμης** (coherence):
 - διασφαλίζει ότι η τιμή της τελευταίας εγγραφής σε μια θέση μνήμης θα γνωστοποιηθεί σε όλους τους τυχόν αναγνώστες
 - δημιουργεί μια καθολικά ενιαία εικόνα για **μία** συγκεκριμένη θέση μνήμης (cache line, πρακτικά)
 - δεν αρκεί:
 - » 2 cache lines A και B μπορεί να είναι μεμονωμένα συνεπείς, αλλά ασυνεπείς σε σχέση με τη σειρά που τροποποιήθηκαν στο πρόγραμμα
- **Συνέπεια μνήμης** (consistency):
 - καθορίζει το *πότε* θα γίνεται ορατή μια εγγραφή
 - δημιουργεί μια καθολικά ενιαία εικόνα για **όλες** τις θέσεις μνήμης, όσον αφορά τις μεταξύ τους τροποποιήσεις
- Απαραίτητη για τη σωστή λειτουργία μηχανισμών συγχρονισμού

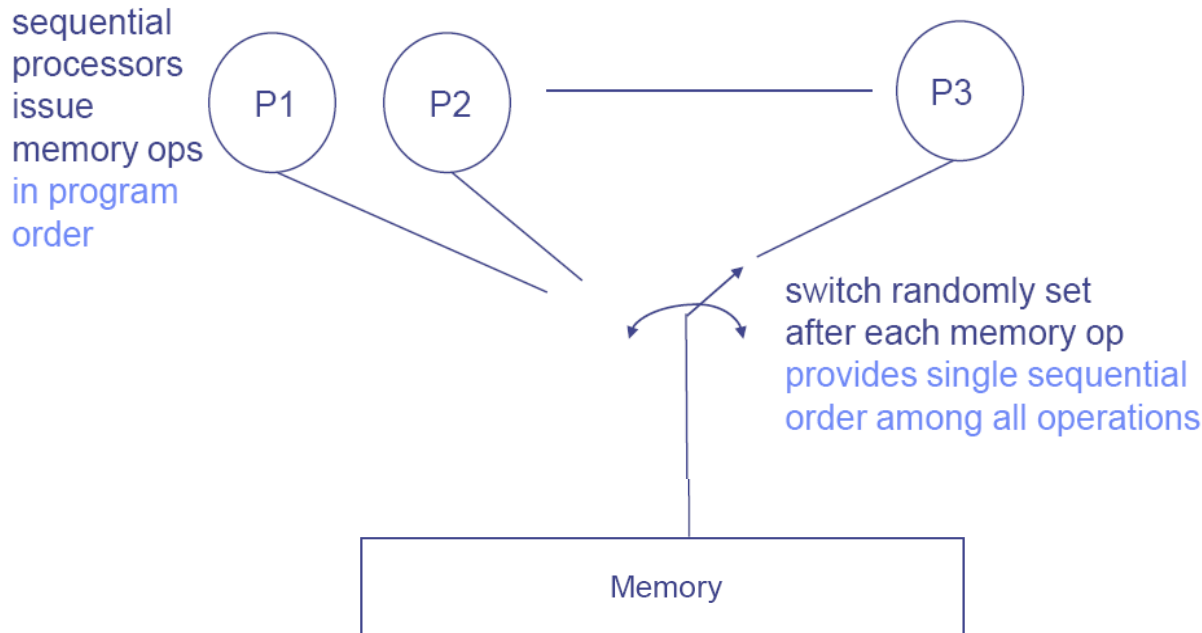
Μοντέλο Συνέπειας Μνήμης

- Περιορίζει τις πιθανές διατάξεις με τις οποίες οι λειτουργίες μνήμης μπορούν να εμφανιστούν ή μια σε σχέση με την άλλη.
- Χωρίς αυτή δεν μπορούμε να πούμε τίποτα για το αποτέλεσμα της εκτέλεσης ενός προγράμματος.
 - Ο *προγραμματιστής* αποφασίζει για την ορθότητα και τα πιθανά αποτελέσματα
 - Ο *σχεδιαστής του συστήματος* περιορίζει πόσο μπορούν να αναδιατάσσονται οι λειτουργίες μνήμης από τον compiler ή το hardware.

A memory consistency (MC) model is a specification of the allowed behavior of multithreaded programs executing with shared memory.

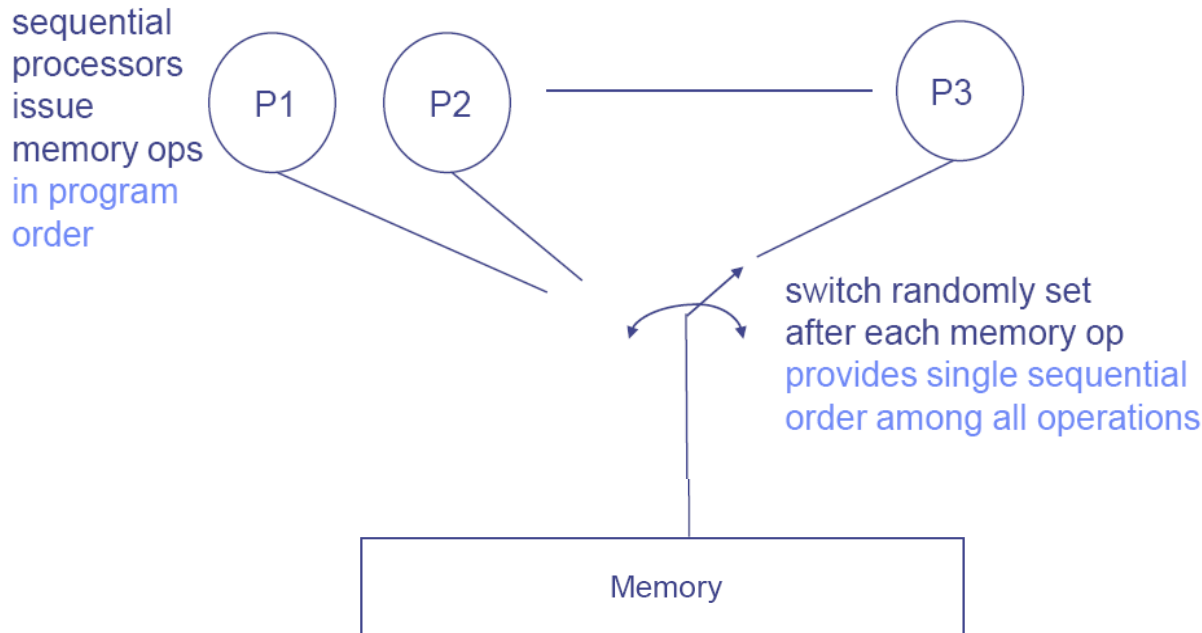
MC is the Architectural Contract

Sequential Consistency



- “A multiprocessor is **sequentially consistent** if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor occur in this sequence in the order specified by its program.” [Lamport, 1979]
- SC = τυχαία μίξη των (εν σειρά) αναφορών των σειριακών προγραμμάτων στους επεξεργαστές

Sequential Consistency (2)



- Σαν να μην υπήρχαν κρυφές μνήμες, παρά μόνο μια μνήμη.
- Κάθε επεξεργαστής δρομολογεί και ολοκληρώνει μια λειτουργία μνήμης σύμφωνα με τη σειρά του προγράμματος
- Η μνήμη ικανοποιεί τις προσπελάσεις σύμφωνα με *κάποια* σειρά.
- Κάθε λειτουργία σε αυτή τη σειρά φαίνεται σα να εκτελείται και να ολοκληρώνεται ατομικά (πριν ξεκινήσουν οι επόμενες).

Sequential Consistency (3)

- Ορισμός *program order*
 - Διαισθητικά η σειρά με την οποία εμφανίζονται οι εντολές στον πηγαίο κώδικα.
 - Η σειρά με την οποία εμφανίζονται οι λειτουργίες μνήμης στην assembly που προκύπτει από απευθείας μετατροπή του πηγαίου κώδικα.
- *Δεν* είναι υποχρεωτικά η ίδια σειρά με αυτή που παράγει ο compiler και εκτελείται στο hardware
 - Ένας optimizing compiler μπορεί να αναδιατάξει τις εντολές του πηγαίου κώδικα.
- Άρα η σειρά του προγράμματος εξαρτάται από το επίπεδο που κοιτάζουμε.
 - Εμείς υποθέτουμε τη *σειρά έτσι όπως τη βλέπει ο προγραμματιστής*.

Sequential Consistency Requirements

- Ανακεφαλαιώνοντας, για την SC έχουμε 2 απαιτήσεις.
- **Req1. Program Order**
 - Οι λειτουργίες μνήμης ενός thread πρέπει να γίνονται ορατές (στους άλλους και στον εαυτό του) με τη σειρά που υπαγορεύει το πρόγραμμα.
- **Req2. Atomicity**
 - Μια λειτουργία μνήμης ολοκληρώνεται προτού κληθεί η επόμενη σύμφωνα με την καθολική σειρά (ανεξάρτητα από το σε ποιο thread ανήκει η επόμενη λειτουργία).
- Ικανές συνθήκες, όχι αναγκαίες!

SC Formalism

All cores insert their loads and stores into the memory order ($<_m$) respecting their program order ($<_p$), regardless of whether they are to the same or different addresses (i.e., $a=b$ or $a \neq b$).

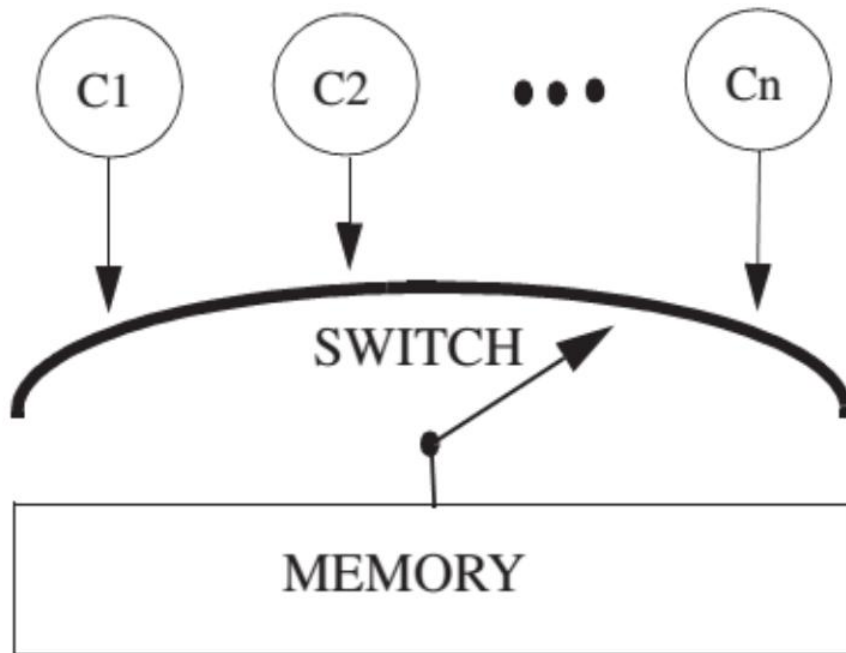
- If $L(a) <_p L(b) \Rightarrow L(a) <_m L(b)$ */* Load→Load */*
- If $L(a) <_p S(b) \Rightarrow L(a) <_m S(b)$ */* Load→Store */*
- If $S(a) <_p S(b) \Rightarrow S(a) <_m S(b)$ */* Store→Store */*
- If $S(a) <_p L(b) \Rightarrow S(a) <_m L(b)$ */* Store→Load */*

Every load gets its value from the last store before it (in global memory order) to the same address:

Value of $L(a)$ = Value of $\text{MAX } <_m \{S(a) \mid S(a) <_m L(a)\}$,
where $\text{MAX } <_m$ denotes “latest in memory order.”

Naïve SC Implementation

- Option 1 – Just run everything on a single core
- Option 2 – memory access through a “switch” serializing memory accesses:



Each core C_i seeks to do its next memory access in its program order $\langle p \rangle$.

The switch selects one core, allows it to complete one memory access, and repeats; this defines memory order $\langle m \rangle$.

Req1 - Program Order (Παράδειγμα)

P1

P2

/* Assume initial values of A and B are 0 */

(1a) A = 1;

(2a) print B;

(1b) B = 2;

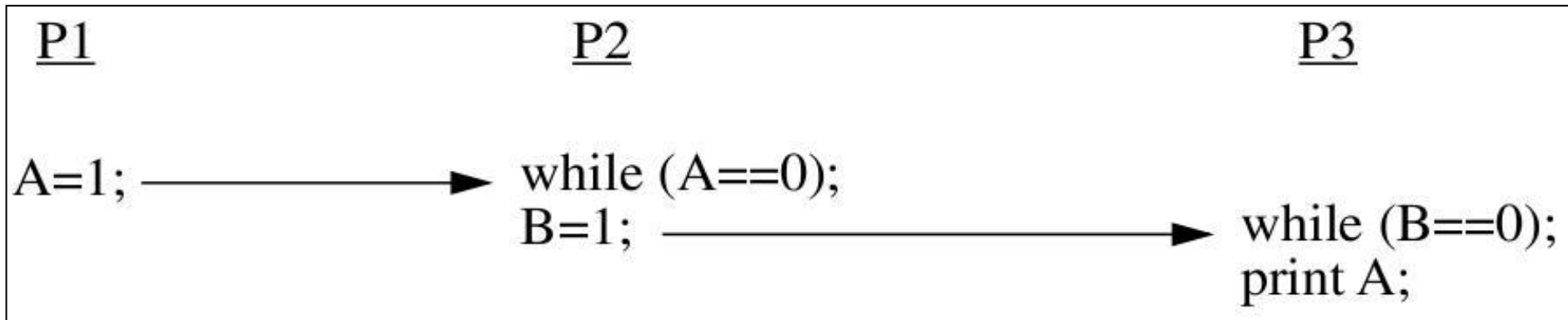
(2b) print A;

- Πιθανά αποτελέσματα για (A,B) :
 - (0,0) σειρά εκτέλεσης: 2a, 2b, 1a, 1b, SC
 - (1,0) σειρά εκτέλεσης: 2a, 1a, 1b, 2b, SC
 - (1,2) σειρά εκτέλεσης: 1a, 1b, 2a, 2b, SC
- Επιτρέπει η SC το (0,2);
 - Σύμφωνα με SC πρέπει $1a \rightarrow 1b$ και $2a \rightarrow 2b$ (program order).
 - Αφού $A = 0$, τότε $2b \rightarrow 1a$.
 - Αφού $B = 2$, $1b \rightarrow 2a$.
 - Υπάρχει κύκλος εξαρτήσεων: $1a \rightarrow 1b \rightarrow 2a \rightarrow 2b \rightarrow 1a$.
 - Δεν υπάρχει μερική διάταξη που να ικανοποιεί όλες τις εξαρτήσεις
=> **μη επιτρεπτό αποτέλεσμα**.

Write atomicity

- Write atomicity
 - Η θέση όπου εμφανίζεται να εκτελείται ένα write σύμφωνα με την καθολική σειρά, **πρέπει** να είναι **ίδια για όλα** τα threads.
- Τίποτα από αυτά που κάνει ένα thread αφού δει την καινούρια τιμή που παράγει μια εγγραφή W δεν πρέπει να γίνει ορατό από τα υπόλοιπα threads πριν δουν και αυτά τη W .
- Ουσιαστικά, επεκτείνουμε την σειριοποίηση εγγραφών που απαιτεί η συνάφεια.
 - Write serialization: Όλες οι εγγραφές σε **μια** τοποθεσία θα πρέπει να εμφανίζονται σε όλα τα threads με την ίδια σειρά
 - Write atomicity: Όλες οι εγγραφές σε **κάθε** τοποθεσία θα πρέπει να εμφανίζονται σε όλα τα threads με την ίδια σειρά

Req2-Write atomicity (Παράδειγμα)



- Έστω ότι επιτρέπουμε στον P2 να προχωρήσει στο B=1 πριν η εγγραφή του A γίνει ορατή από τον P3.
- Μπορεί ο P3 να διαβάσει την παλιά τιμή του A και την καινούρια του B;
- Παραβίαση της SC!

Sequential Consistency SC (Example 1)

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag; B1: if (r1 $\neq$ SET) goto L1; L2: Load r2 = data;	/* Initially, data = 0 & flag $\neq$ SET */ /* L1 & B1 may repeat many times */

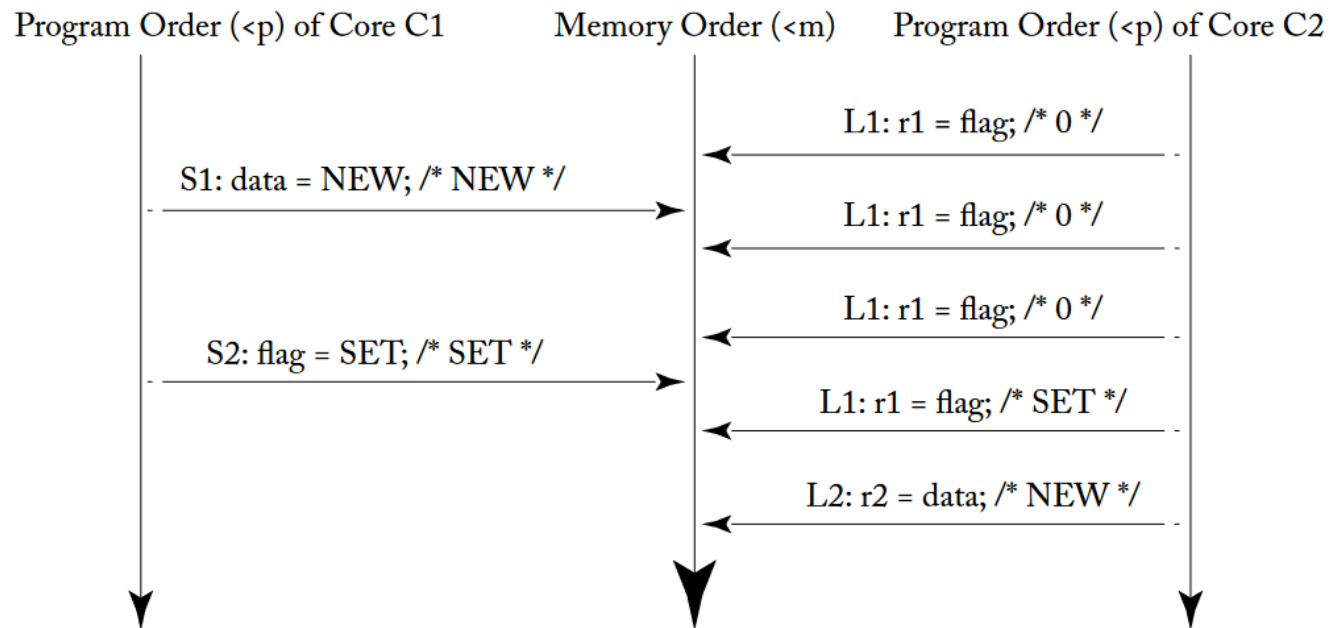


Figure 3.2: A sequentially consistent execution of Table 3.1's program.

Sequential Consistency SC (Example 1)

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag; B1: if (r1 ≠ SET) goto L1; L2: Load r2 = data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

r2=NEW seems to be the only accepted result. But r2=0 in a modern hardware is also possible!

Hardware can make r2 get the value 0 by reordering core C1's stores S1 and S2. The hardware may reorder stores-loads e.g., with a write buffer that is not first-in-first-out.

With the reordering of S1 and S2, the execution order may be S2, L1, L2, S1:

Cycle	Core C1	Core C2	Coherence State of Data	Coherence State of Flag
1	S2: Store flag = SET		Read-only for C2	Read-write for C1
2		L1: Load r1=flag	Read-only for C2	Read-only for C2
3		L2: Load r2=data	Read-only for C2	Read-only for C2
4	S1: Store data = NEW		Read-write for C1	Read-only for C2

Source: "A Primer on Memory Consistency and Cache Coherence" - 2nd edition, Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood
https://pages.cs.wisc.edu/~markhill/papers/primer2020_2nd_edition.pdf

How a Core Might Reorder Memory Access (Example 2)

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag; B1: if (r1 ≠ SET) goto L1; L2: Load r2 = data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

Store-store reordering. Two stores may be reordered if a core has a **non-FIFO** write buffer that lets stores depart in a different order than the order in which they entered. *This might occur if the first store misses in the cache while the second hits or if the second store can coalesce with an earlier store (i.e., before the first store).* These reorderings are possible even if the core executes all instructions in program order. Reordering stores to different memory addresses has no effect on a single-threaded execution.

In the multithreaded example above, reordering Core C1's stores allows Core C2 to see flag as SET before it sees the store to data. **Note that the problem is not fixed even if the write buffer drains into a perfectly coherent memory hierarchy.** Coherence will make all caches invisible, but the stores are already reordered.

How a Core Might Reorder Memory Access

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag; B1: if (r1 ≠ SET) goto L1; L2: Load r2 = data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

Load-load reordering. Modern dynamically scheduled cores may execute instructions out of program order. In the above example, Core C2 could execute loads L1 and L2 out of order. Considering only a single-threaded execution, this reordering seems safe because L1 and L2 are to different addresses. However, reordering Core C2's loads behaves the same as reordering Core C1's stores; if the memory references execute in the order L2, S1, S2, and L1, then r2 is assigned 0. This scenario is even more plausible if the branch statement B1 is elided, so no control dependence separates L1 and L2.

How a Core Might Reorder Memory Access

Table 3.3: Can both r1 and r2 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */

Load-store and store-load reordering. Out-of-order cores may also reorder loads and stores (to different addresses) from the same thread:

A. Reordering an earlier load with a later store (a load-store reordering) can cause many incorrect behaviors, such as loading a value after releasing the lock that protects it (if the store is the unlock operation).

B. The example above (table 3.3) illustrates the effect of reordering an earlier store with a later load (a store-load reordering). Reordering Core C1's accesses S1 and L1 and Core C2's accesses S2 and L2 allows the counterintuitive result that both r1 and r2 are 0.

Note that **store-load reorderings may also arise due to local bypassing in the commonly implemented FIFO write buffer, even with a core that executes all instructions in program order.**

A reader might assume that hardware should not permit some or all of these behaviors, but without a better understanding of what behaviors are allowed, it is hard to determine a list of what hardware can and cannot do.

SC (Example 2)

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */

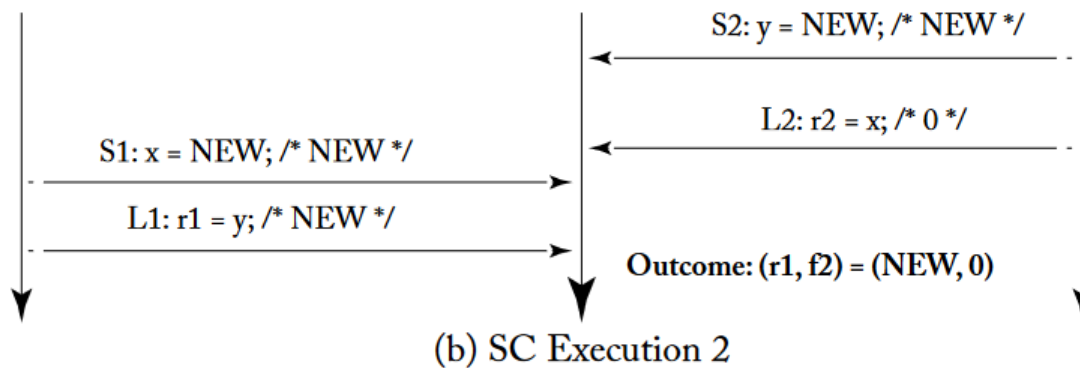
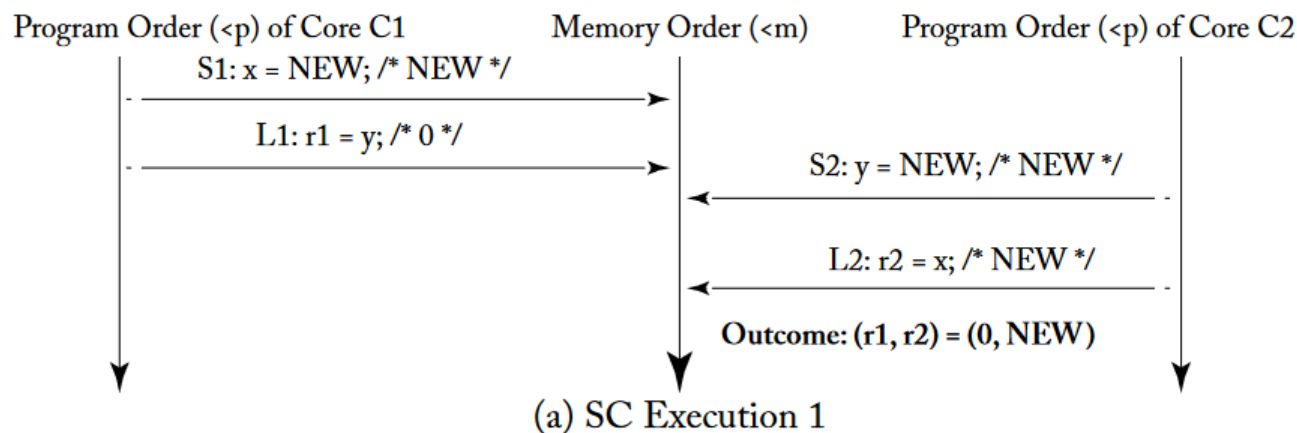
Is $(r1, r2) = (0, 0)$ possible?

- $(r1, r2) = (0, \text{NEW})$ for execution S1, L1, S2, then L2. **Allowed SC.**
- $(r1, r2) = (\text{NEW}, 0)$ for S2, L2, S1, and L1. **Allowed by SC.**
- $(r1, r2) = (\text{NEW}, \text{NEW})$, e.g., for S1, S2, L1, and L2. **Allowed by SC.**
- $(r1, r2) = (0, 0)$, e.g. for L1, L2, S1, S2. **Result forbidden by SC.**

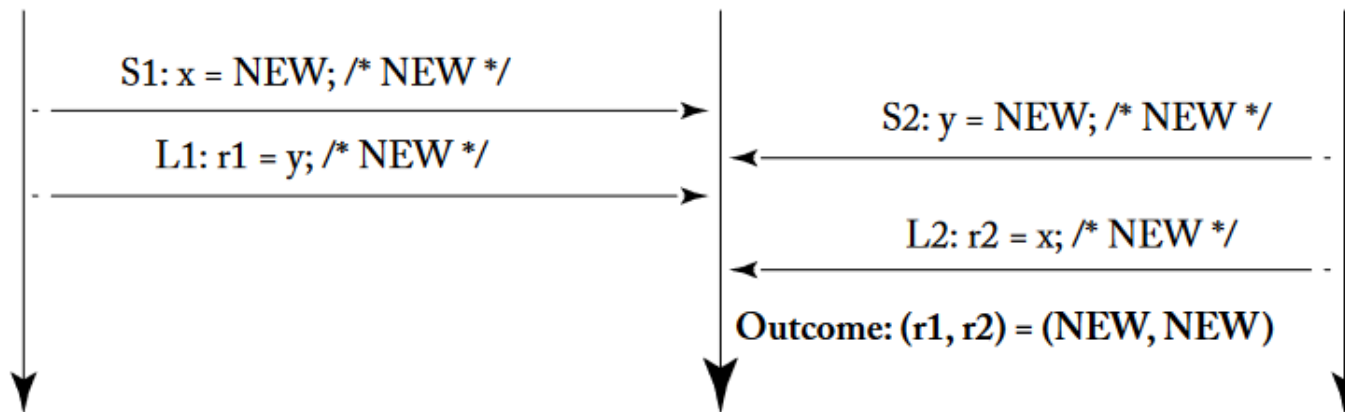
all of these executions satisfy cache coherence, even $(r1, r2) = (0, 0)$.

Surprisingly, most real hardware, e.g., x86 systems from Intel and AMD, also allows $(r1, r2) = (0, 0)$ because it uses first-in–first-out (FIFO) write buffers to enhance performance.

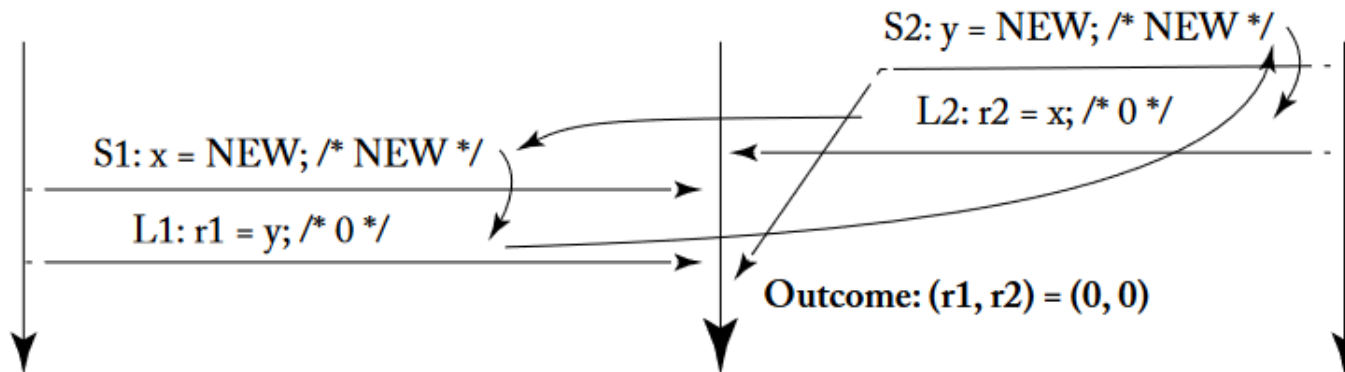
Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */



Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */



(c) SC Execution 3



(d) NOT an SC Execution

Προβλήματα SC: Hardware

- Πολύ λίγα συστήματα υλοποιούν SC
 - Ούτε οι x86, ούτε οι ARM
- Μειωμένη απόδοση
 - Αναμονή για ολοκλήρωση ενός store πριν την δρομολόγηση του επόμενου
- Δύσκολη η υλοποίηση της όταν χρησιμοποιούνται διάφορες τεχνικές βελτίωσης επίδοσης του επεξεργαστή
 - OOO execution: Loads out-of-order σε σχέση με άλλα loads και stores
 - Caching: Πως θα δούνε όλοι οι επεξεργαστές την ίδια καθολική σειρά;
- Πολύπλοκο hardware
 - π.χ. MIPS R10K: “speculatively issue loads but squash if memory inconsistency with later-issued store is discovered”

Προβλήματα SC: Software

- Ο compiler μπορεί να **αναδιατάξει** εντολές πρόσβασης στη μνήμη
 - Instruction scheduling: Μεταφορά ενός Load πριν από Store αν πρόκειται για διαφορετικές διευθύνσεις
- Ο compiler μπορεί να **διαγράψει** εντολές πρόσβασης στη μνήμη
 - Register allocation: Αν η τιμή είναι cached σε ένα καταχωρητή δεν χρειάζεται να ελεγχθεί η μνήμη
- Οι βελτιστοποιήσεις του μεταφραστή είναι αναγκαίες για λόγους απόδοσης!

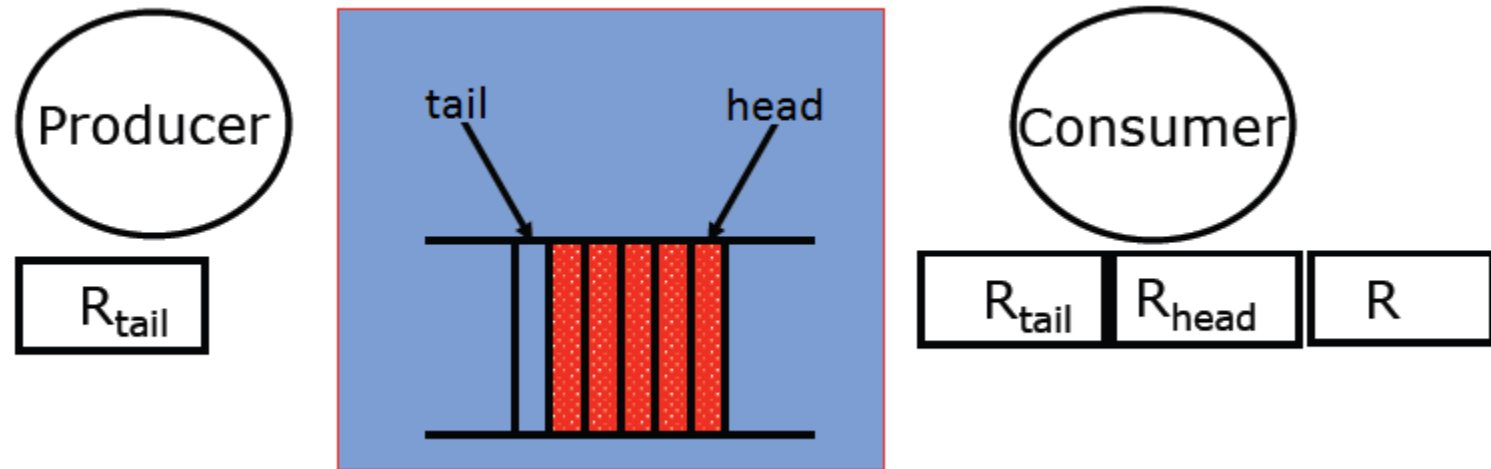
Επίλυση προβλημάτων SC

- Χρειαζόμαστε global order για όλες τις λειτουργίες και όλους τους επεξεργαστές;
- Θα αρκούσε να διασφαλίζουμε global order μόνο για όλα τα stores;
- Θα αρκούσε να διασφαλίζουμε global order μόνο όπου χρειάζεται συγχρονισμός; (τι σημαίνει συγχρονισμός και πως ξέρω αν χρειάζεται;)
- Λύση → **Relaxed Memory Models**

Relaxed Memory Models

- Η SC είναι πολύ αυστηρή → Τα relaxed models δεν υποστηρίζουν όλες τις απαιτήσεις της SC.
- Ο προγραμματιστής πρέπει να εισάγει στον κώδικα του ότι απαιτήσεις/εξαρτήσεις χρειάζονται και δεν υποστηρίζονται από το μοντέλο μνήμης.
 - Local vs shared memory space!
 - Fence Instructions (sync or memory barriers)
 - Atomic memory instructions

Παράδειγμα: Relaxed Consistency με Fences



Producer posting Item x:

Load $R_{tail}, (tail)$

Store $(R_{tail}), x$

Fence_{SS}

$R_{tail} = R_{tail} + 1$

Store $(tail), R_{tail}$

εγγυάται ότι ο tail pointer δε θα ανανεωθεί πριν την εγγραφή του x

Consumer:

Load $R_{head}, (head)$

spin: Load $R_{tail}, (tail)$

if $R_{head} == R_{tail}$ goto spin

Fence_{LL}

Load $R, (R_{head})$

$R_{head} = R_{head} + 1$

Store $(head), R_{head}$

consume(R)

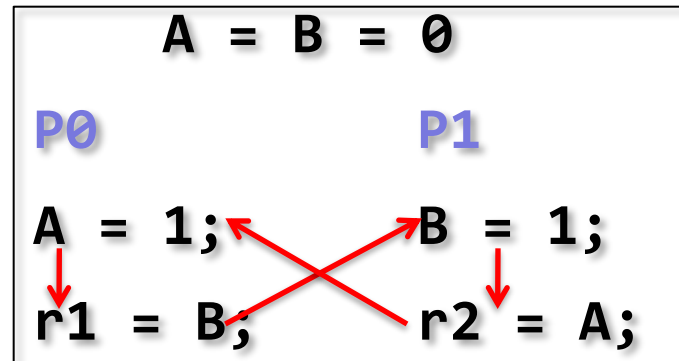
εγγυάται ότι ο R δε θα φορτωθεί πριν την εγγραφή του x

Relaxed Consistency Models (2)

- Μπορούμε να οργανώσουμε τα μοντέλα σε 3 διαφορετικές κατηγορίες
1. Επιτρέπεται σε ένα Load να ολοκληρωθεί προτού ολοκληρωθεί κάποιο προηγούμενο (σύμφωνα με το program order) Store.
 - Total Store Order (TSO), Processor Consistency (PC) (e.g., IBM 370, Sparc TSO, Intel IA-32)
 2. Επιτρέπεται και σε ένα Store να προσπεράσει κάποιο προηγούμενο (σύμφωνα με το program order) Store.
 - Partial Store Order (PSO) (e.g., Sparc PSO)
 3. Επιτρέπεται σε ένα Load ή ένα Store να προσπεράσει κάποια προηγούμενη (σύμφωνα με το program order) λειτουργία μνήμης, είτε Load είτε Store.
 - Weak Ordering (WO), Release Consistency (RC), Relaxed Memory Order (RMO) (e.g., Sparc RMO, Alpha, PowerPC)

Relaxing Write-to-Read Order

- Αντιμετώπιση της καθυστέρησης για writes που κάνουν miss στην L1 cache.
- Όσο το write είναι στο write buffer, ο επεξεργαστής ολοκληρώνει reads που κάνουν hit στην cache.
- Καταργείται η συνθήκη της SC ότι όλες οι αναφορές πραγματοποιούνται στη σειρά προγράμματος



- επιτρέπει $r1 == r2 == 0$ (δεν το επιτρέπει η SC)

Relaxing Write-to-Read & Write-to-Write Order

- Κίνητρο: Περαιτέρω μείωση της καθυστέρησης εξαιτίας ενός write miss και βελτίωση της επικοινωνίας μεταξύ των επεξεργαστών κάνοντας ορατές τις καινούριες τιμές νωρίτερα.
- Επιτρέπεται το merging πολλαπλών writes που βρίσκονται στο **write buffer**.
- Τα writes μπορεί να γίνουν ορατά εκτός σειράς!

Γιατί να μην χαλαρώσουμε όλους τους περιορισμούς;

```
/* initially all 0 */
```

P1

```
A = 1;
```

```
B = 1;
```

```
flag = 1;
```

P2

```
while (flag == 0); /* spin */
```

```
  r1 = A;
```

```
  r2 = B;
```

- OK αν μπορούμε να αναδιατάξουμε τα ζεύγη “A=1/B=1” ή “r1=A/r2=B”
 - μέσω OoO επεξεργασιών, non-FIFO write buffers, καθυστερήσεις στο δίκτυο διασύνδεσης, κ.λπ.
- Όμως ο προγραμματιστής στηρίζεται στα εξής για την ορθότητα του προγράμματος:
 - “A=1/B=1” προτού “flag=1”
 - “flag!=0” προτού “r1=A/r2=B”

Γιατί να χαλαρώσουμε όλους τους περιορισμούς;

```
/* initially all 0 */
```

P1

```
X = 1;
```

```
Y = 1;
```

```
flag2 = 1;
```

P2

```
while (flag == 0); /* spin */
```

```
  r1 = A;
```

```
  r2 = B;
```

- Σε αυτό το σενάριο *δεν υπάρχει επικοινωνία*. Γιατί να μην «τρέξει» γρήγορα, αλλά να καθυστερήσει από περιορισμούς;

Relaxing All Memory Orders

```
/* initially all 0 */
```

P1

```
A = 1;
```

```
B = 1;
```

```
SYNCH flag = 1;
```

P2

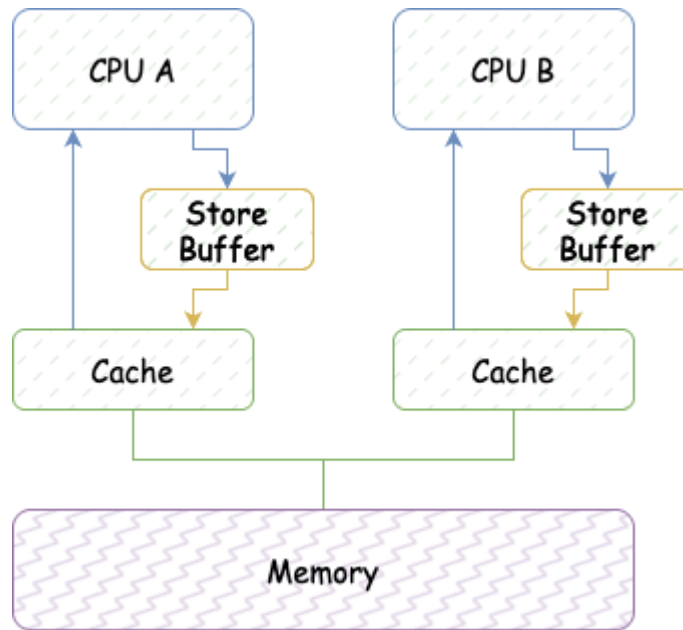
```
while (SYNCH flag == 0);
```

```
r1 = A;
```

```
r2 = B;
```

- Κίνητρο: Όταν μας ενδιαφέρει η σειρά των λειτουργιών, τα παράλληλα προγράμματα χρησιμοποιούν συγχρονισμό. Επομένως, μπορούμε χαλαρώσουμε όλους τους περιορισμούς για τις λειτουργίες πριν και μετά τις λειτουργίες συγχρονισμού.
- Weak Ordering (WO): Ξεχωρίζει μεταξύ “κανονικών” λειτουργιών και λειτουργιών συγχρονισμού.
- Release Consistency (RC): Επεκτείνει το WO, διακρίνοντας τις λειτουργίες συγχρονισμού σε *acquire* και *release* (επιτρέποντας καλύτερη επικάλυψη των λειτουργιών)

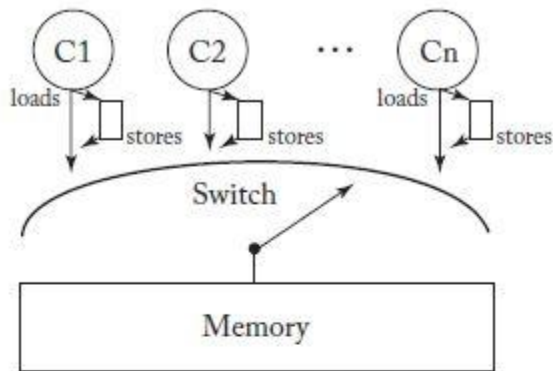
The Write Buffer Advantage



Processors use write buffers to hold committed stores until the memory system can process them.
Hides latency of store misses.
Avoids stalling on common store operations.
Architecturally invisible to a single-core, but visible to multicore.

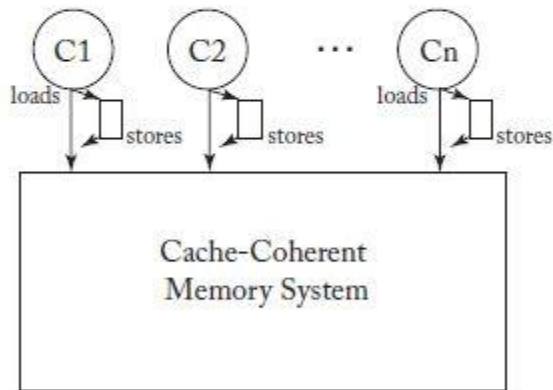
Total Store Order (TSO)

- Was first introduced by SPARC
- ***Matches*** the memory consistency model of x86.
- RISC-V also supports a TSO extension, RVTSO, in part to aid porting of code originally written for x86 or SPARC architectures.



This implementation is the same as for Figure 3.4, except that each core C_i has a FIFO write buffer that buffers stores until they go to memory.

(a) A TSO implementation using a switch



This TSO implementation replaces the switch above with a cache-coherent memory system in a manner analogous to what was done for SC.

(b) A TSO implementation using cache coherence

Figure 4.4: Two TSO implementations.

TSO motivation

- Processors use *write buffers* to hold committed stores until the memory system can process them.
- A store enters the write buffer when the store commits, and a store exits the write buffer when the block to be written is in the cache in a read–write coherence state.
- A store can enter the write buffer before the cache has obtained read–write coherence permissions
 - The write buffer hides the latency of a store miss.
 - Stores are common, being able to avoid stalling on most of them is an important benefit.
- For a single-core processor
 - a write buffer can be made invisible by ensuring that a load returns the value of the most recent store even if one or more stores to are in the write buffer.
 - This can be done by *bypassing* the value of the most recent store as determined by program order,
 - Or by stalling a load if a store to the same address is in the write buffer.

Total Store Order (TSO) – Motivation

- When building a multicore processor, it seems natural to use multiple cores, each with its own bypassing write buffer
 - Assume that the write buffers continue to be architecturally invisible as before.
 - This assumption is wrong!

Core C_1	Core C_2
S1: $x = \text{NEW}$	S2: $y = \text{NEW}$
L1: $r1 = y$	L2: $r2 = x$

- Example:
 - Core C_1 executes store S1, buffers the NEW value in its write buffer.
 - Core C_2 executes store S2, buffer the NEW value in its write buffer.
 - Both cores perform L1 and L2 → obtain the old values!
 - Finally, both cores' write buffers update memory with NEW

Total Store Order (TSO) – Motivation

- The option chosen by SPARC and x86* was to abandon SC.
- Instead, implement an MC model that allows use of a **FIFO** write buffer in each core.
- The new model, TSO, allows the outcome “ $(r1, r2) = (0, 0)$ ”!
- Behaves like SC for many programming idioms and is well defined in all cases, but can be surprising.

* Not entirely TSO

The ‘relaxation’ behind TSO

As execution proceeds, SC requires that each core preserves the program order of its loads and stores for all four combinations of consecutive operations:

- Load → Load
- Load → Store
- Store → Store
- **Store → Load** /* Included for SC but omitted for TSO */

TSO includes the first three constraints **but not the fourth**. This omission does not matter for most programs

TSO Formalism

All cores insert their loads and stores into the order $<_m$ respecting their program order, regardless of whether they are to the same or different addresses (i.e., $a=b$ or $a \neq b$).

- If $L(a) <_p L(b) \Rightarrow L(a) <_m L(b)$ /*Load→Load*/
- If $L(a) <_p S(b) \Rightarrow L(a) <_m S(b)$ /*Load→Store*/
- If $S(a) <_p S(b) \Rightarrow S(a) <_m S(b)$ /*Store→Store*/
- If ~~$S(a) <_p L(b) \Rightarrow S(a) <_m L(b)$~~ /*Store→Load*/: Enable FIFO write buffer

Every load gets its value from the last store before it (in global memory order) to the same address:

$$\text{Value of } L(a) = \text{Value of MAX } <_m \{S(a) \mid S(a) <_m L(a) \text{ or } S(a) <_p L(a)\}$$

($<_m$ “last in memory order.”, $<_p$ “last in program order”)

The value of a load is the value of the last store to the same address that is either (a) before it in memory order or (b) before it in program order (but possibly after it in memory order)

Option (b) taking precedence - (i.e., write buffer bypassing overrides the rest of the memory system)

TSO behaves as SC for many programs

Table 4.2: Should r2 always be set to NEW?

Core C1	Core C2	Comments
S1: Store data = NEW; S2: Store flag = SET;	L1: Load r1 = flag B1: if (r1 ≠ SET) goto L1; L2: Load r2=data;	/* Initially, data = 0 & flag ≠ SET */ /* L1 & B1 may repeat many times */

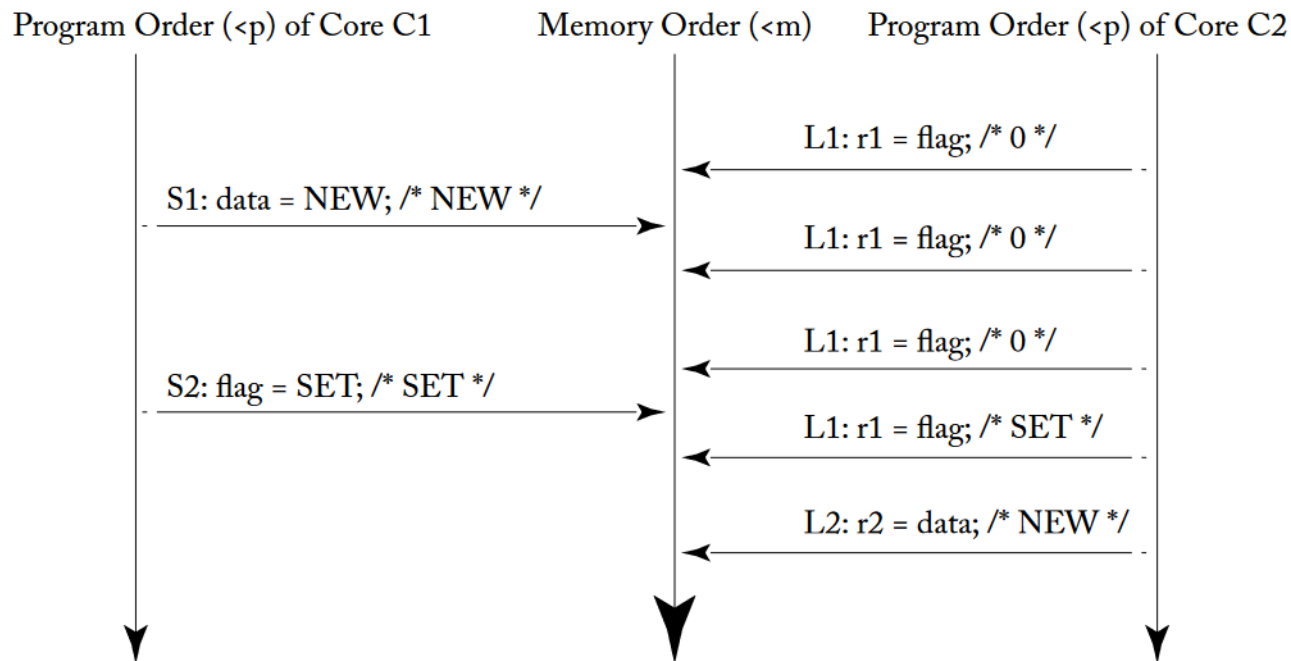
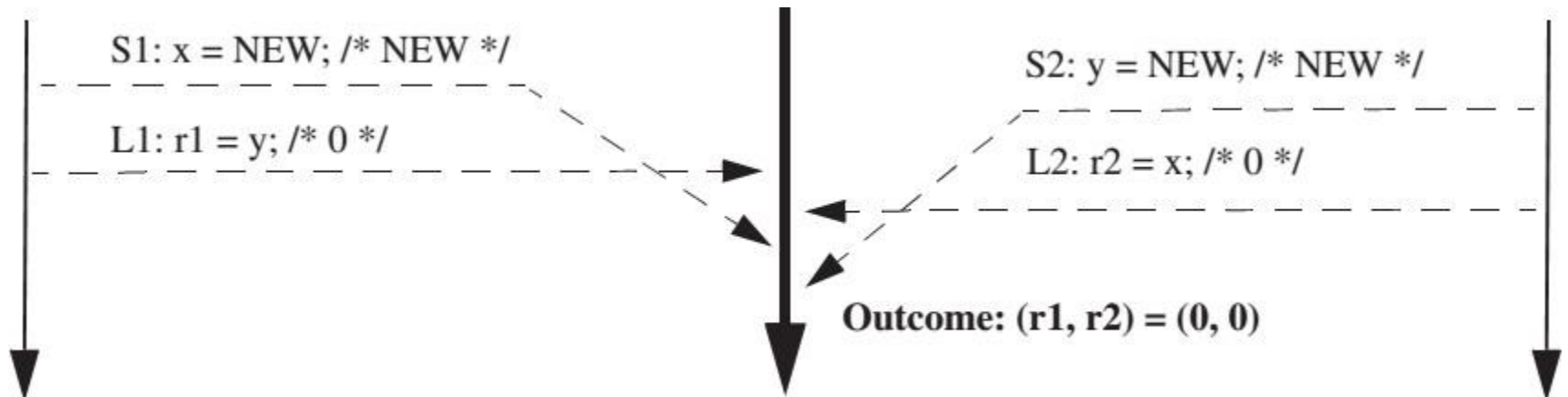


Figure 4.1: A TSO execution of Table 4.2's program.

TSO Example

Core C_1	Core C_2
S1: $x = \text{NEW}$	S2: $y = \text{NEW}$
L1: $r1 = y$	L2: $r2 = x$



(d) TSO Execution, but NOT an SC Execution

However TSO allows also non SC executions:

Write Buffers when Multicores

Assume a multicore processor with in-order cores, where each core has a single-entry write buffer and executes the code in the following sequence

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */

Is $(r1, r2) = (0, 0)$ possible?

- Core C1 executes store S1, but buffers the newly stored NEW value in its write buffer.
- Core C2 executes store S2 and holds the newly stored NEW value in its write buffer.
- Next, both cores perform their respective loads, L1 and L2, and obtain the old values of 0.
- Finally, both cores' write buffers update memory with the newly stored values NEW.

The net result is that $(r1, r2) = (0, 0)$. **Result forbidden by SC.**

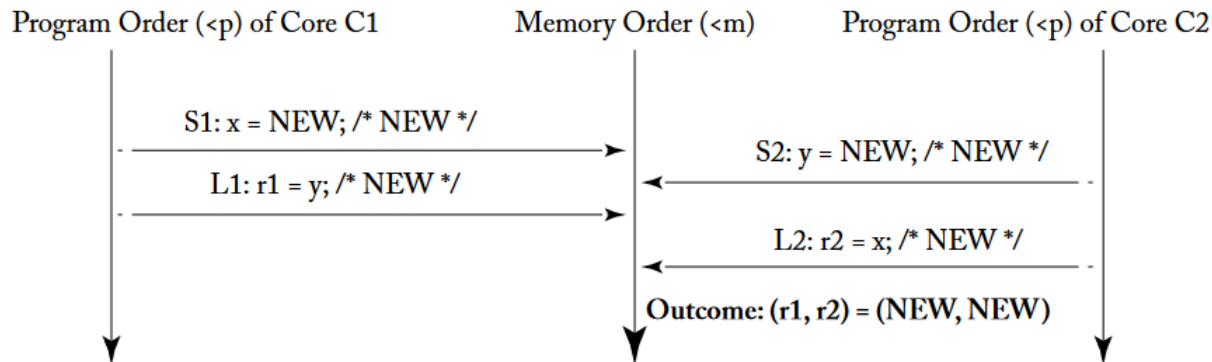
Without write buffers, the hardware is SC, but with write buffers, it is not, making write buffers architecturally visible in a multicore processor.

Store → Load violation:

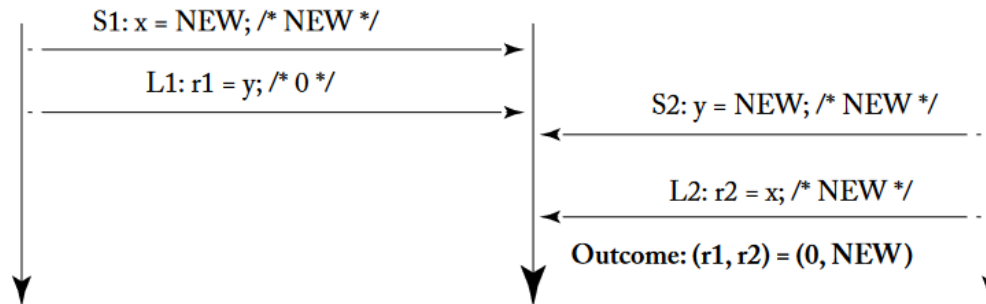
Store S1 happens after L1 (store-load reordering possible under TSO)

TSO

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */



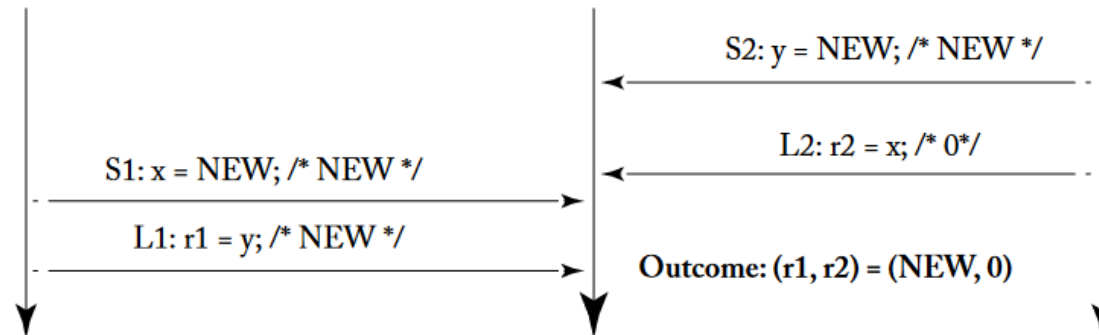
(a) TSO and SC Execution 1



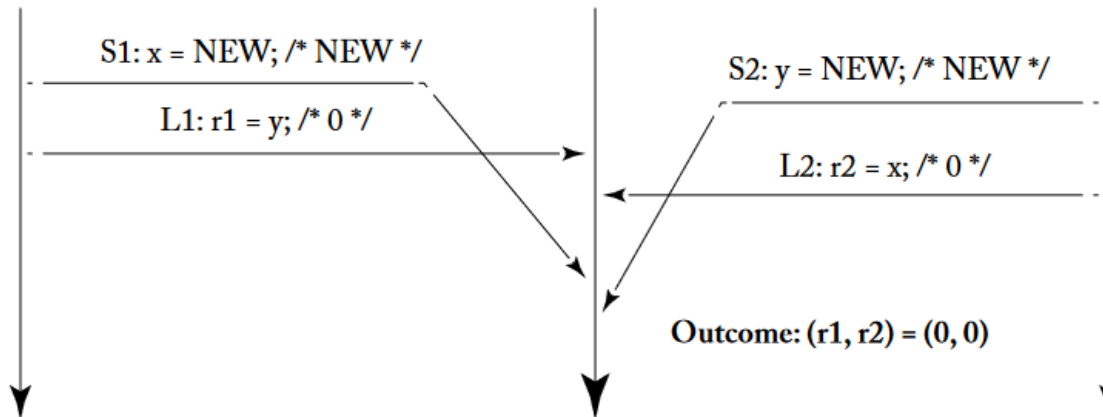
(b) TSO and SC Execution 2

TSO

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = y;	S2: y = NEW; L2: r2 = x;	/* Initially, x = 0 & y = 0 */



(c) TSO and SC Execution 3



(d) TSO Execution, but NOT an SC Execution

Source: "A Primer on Memory Consistency and Cache Coherence" - 2nd edition, Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood

https://pages.cs.wisc.edu/~markhill/papers/primer2020_2nd_edition.pdf

TSO behavior

- TSO behaves the same as SC for common programming idioms that follow:
- C1 loads and stores to memory locations $D1, \dots, Dn$ (often data),
- C1 stores to F (often a synchronization flag) to indicate that the above work is complete,
- C2 loads from F to observe the above work is complete (sometimes spinning first and often using a RMW instruction), and
- C2 loads and stores to some or all of the memory locations $D1, \dots, Dn$.

TSO does allow some non-intuitive execution results!

Let's modify the previous example with local copies of x and y to r1 and r3 respectively:

Table 4.3: Can r1 or r3 be set to 0?

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = x; L2: r2 = y;	S2: y = NEW; L3: r3 = y; L4: r4 = x;	/* Initially, x = 0 & y = 0 */ /* Assume r2 = 0 & r4 = 0 */

Can r1 or r3 be set to 0? **NO**

TSO does allow some non-intuitive execution results!

Core C1	Core C2	Comments
S1: x = NEW;	S2: y = NEW;	/* Initially, x = 0 & y = 0 */
L1: r1 = x;	L3: r3 = y;	
L2: r2 = y;	L4: r4 = x;	/* Assume r2 = 0 & r4 = 0 */

Table above illustrates a modified version of the program in which cores C1 and C2 make local copies of x and y, respectively. Many programmers might assume that if both r2 and r4 equal 0, then r1 and r3 should also be 0 because the stores S1 and S2 must be inserted into memory order after the loads L2 and L4.

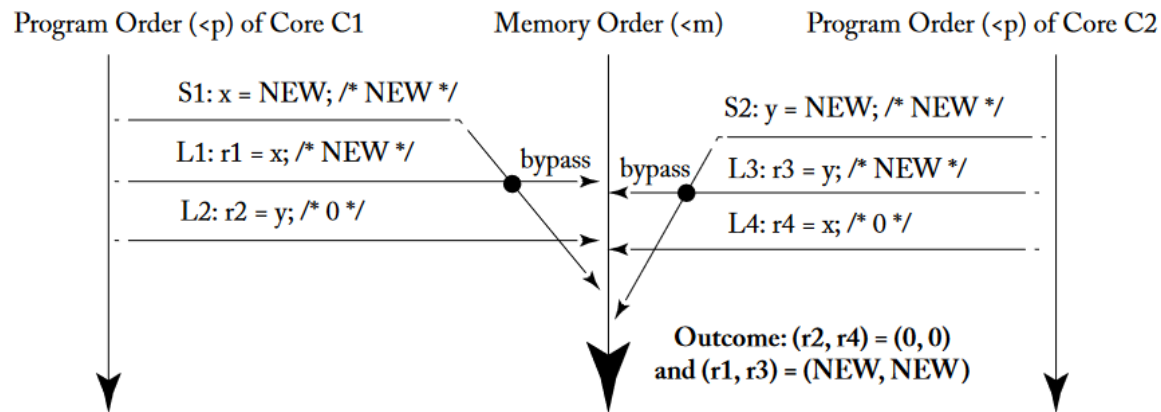


Figure 4.3: A TSO execution of Table 4.3's program (with "bypassing").

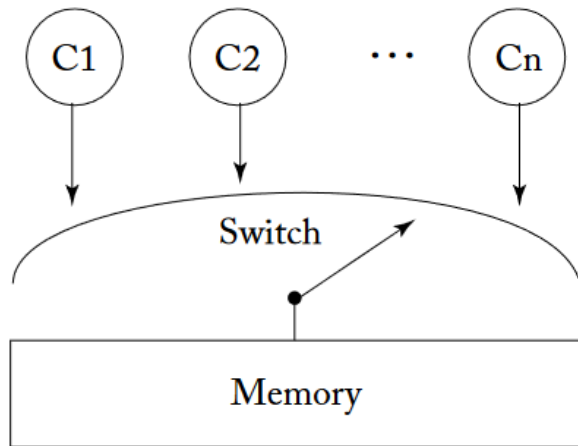
However, Figure 4.3 illustrates an execution that shows r1 and r3 bypassing the value NEW from the **per-core write buffers**. In fact, **to preserve single-thread sequential semantics, each core must see the effect of its own store in program order**, even though the store is not yet observed by other cores. **Thus, under all TSO executions, the local copies r1 and r3 will always be set to the NEW value.**

TSO does allow some non-intuitive execution results!

Core C1	Core C2	Comments
S1: x = NEW; L1: r1 = x; L2: r2 = y;	S2: y = NEW; L3: r3 = y; L4: r4 = x;	/* Initially, x = 0 & y = 0 */ /* Assume r2 = 0 & r4 = 0 */

The value of a load is the value of the last store to the same address that is either (a) **before it in memory order** or (b) **before it in program order** (but possibly after it in memory order), with option (b) taking precedence (i.e., write buffer bypassing overrides the rest of the memory system)

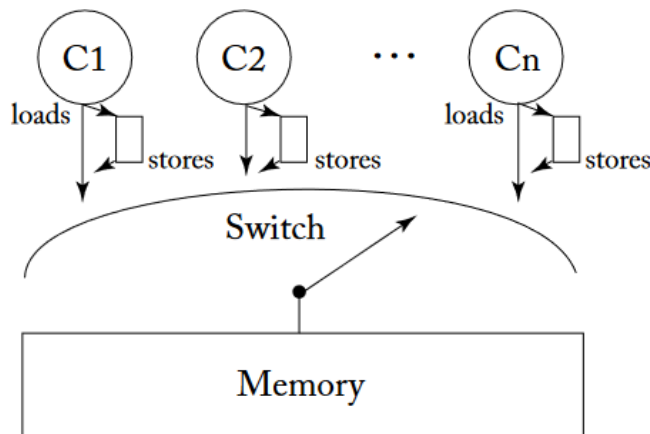
SC vs TSO abstract model



Each core C_i seeks to do its next memory access in its program order $\langle p \rangle$.

The switch selects one core, allows it to complete one memory access, and repeats; this defines memory order $\langle m \rangle$.

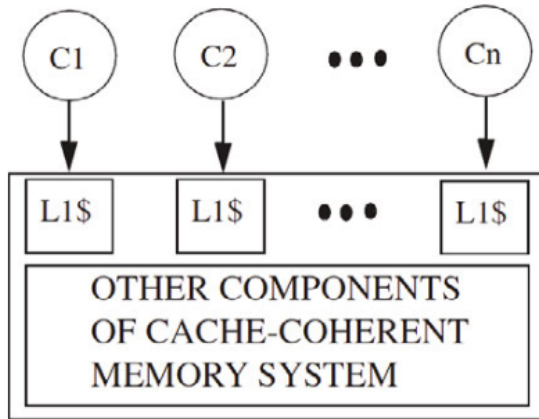
Figure 3.4: A simple SC implementation using a memory switch.



This implementation is the same as for Figure 3.4, except that each core C_i has a FIFO write buffer that buffers stores until they go to memory.

(a) A TSO implementation using a switch

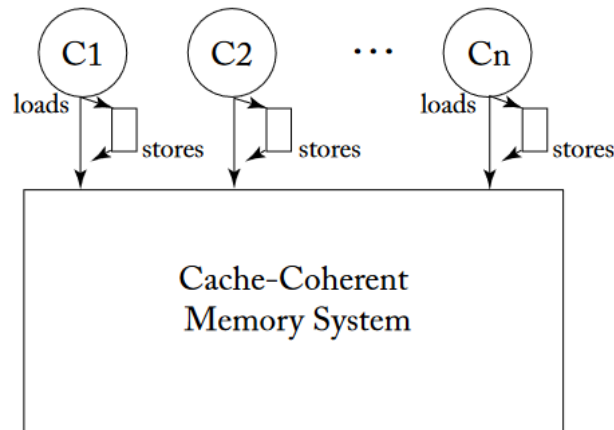
SC vs TSO abstract model with cache coherent memory system



Same as above

Same as above, but cores can concurrently complete accesses to blocks with sufficient (L1) cache coherence permission, because such accesses must be non-conflicting (to different blocks or all loads) and may be placed into memory order in any logical order.

(b) Memory system with L1 caches exposed

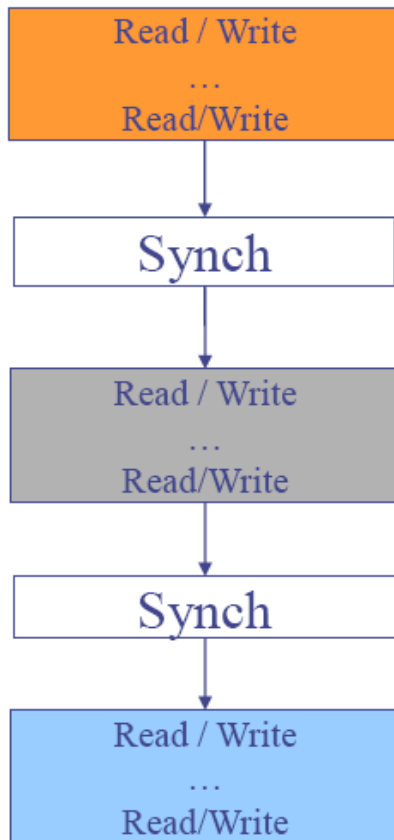


This TSO implementation replaces the switch above with a cache-coherent memory system in a manner analogous to what was done for SC.

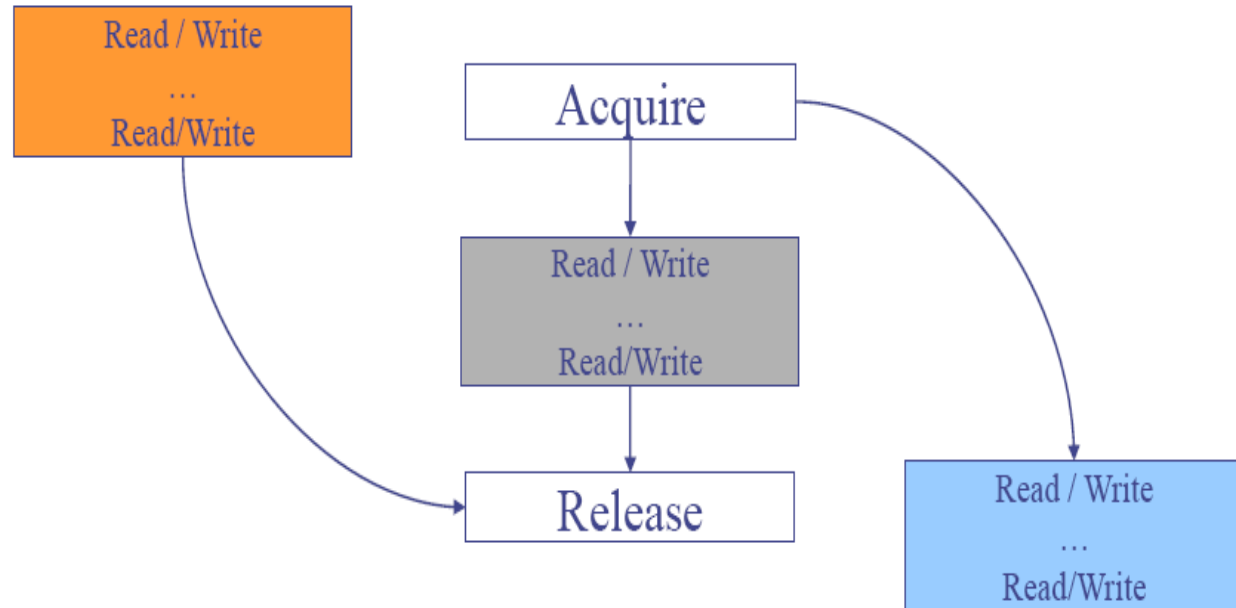
(b) A TSO implementation using cache coherence

Weak Ordering vs Release Consistency

WO



RC



Release Consistency:

1. Before an access to a shared variable is performed, all previous acquires by this processor must have completed.
2. Before a release is performed, all previous reads and writes by this process must have completed.

Παράδειγμα: Sparc V9 memory fences

- #LoadLoad
- #StoreLoad
- #LoadStore
- #StoreStore
- Logical or-ed combinations possible
- #XY = “**All X operations** that appear before the memory fence in program order **complete before any Y operations** that follow after the memory fence in program order.”
- (+) Ευελιξία όσον αφορά την βέλτιστη εκμετάλλευση του εκάστοτε relaxed consistency model για μέγιστη απόδοση
- (-) Προγραμματιστικά δύσκολη + ζητήματα μεταφερσιμότητας ανάμεσα σε διαφορετικά μοντέλα

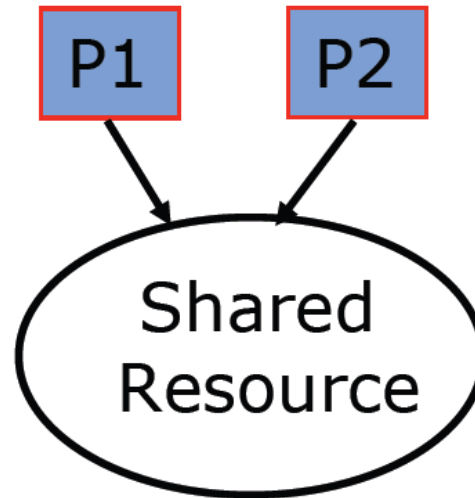
Relaxed Consistency Models (3)

- Hardware optimizations
 - Η χρήση buffers επιτρέπει στον επεξεργαστή να μη κάνει stall εφόσον οι τοπικές εξαρτήσεις δεδομένων διατηρούνται.
 - Το hardware πρέπει να μπορεί να ξεχωρίσει ποιες προσβάσεις στη μνήμη γίνονται για συγχρονισμό και ποιες όχι.
- Software optimizations
 - Αναδιάταξη των εντολών μεταξύ των σημείων συγχρονισμού.
 - Επιτρέπονται compiler optimizations όπως register allocation.
 - Το πρόγραμμα πρέπει να περιέχει τα κατάλληλα annotations και τυχόν memory races να επιλύονται με χρήση συγχρονισμού.
- Ποιο είναι το πρόβλημα με την RC;
 - Τα παράλληλα προγράμματα θα έχουν απροσδιόριστη συμπεριφορά αν εκτελεστούν σε ένα consistency model διαφορετικό από αυτό για το οποίο έχουν γραφτεί

Consistency Models

- Sequential Consistency
 - Οι λειτουργίες μνήμης πρέπει να ολοκληρώνονται με βάση τη σειρά του προγράμματος.
 - Η καθυστέρηση μπορεί να μειωθεί ίσως με χρήση speculation.
 - Δεν επιτρέπονται τα περισσότερα compiler optimizations
 - x86, ARM : not SC.
- TSO & PC
 - Επιτρέπεται η χρήση write buffers.
 - Δεν επιτρέπονται και πάλι τα περισσότερα compiler optimizations.
- WO & RC
 - Επιτρέπεται η χρήση read και write buffers.
 - Επιτρέπονται compiler optimizations μεταξύ των σημείων συγχρονισμού.
 - Το πρόγραμμα πρέπει να περιέχει τα κατάλληλα annotations για να εκτελεστεί σωστά.
 - Η RC έχει υιοθετηθεί στο memory model των C/C++ και Java. Αν όλα τα data accesses προστατεύονται από συγχρονισμό συμπεριφέρεται σαν SC.

Η ανάγκη για συγχρονισμό



- Αρκούν τα coherence protocols και τα consistency models για να μας εξασφαλίσουν σωστή σημασιολογία στην παράλληλη εκτέλεση ενός προγράμματος;

Η ανάγκη για συγχρονισμό (2)

Processor 0

0: `addi r1,accts,r3`

1: `ld 0(r3),r4`

2: `blt r4,r2,6`

3: `sub r4,r2,r4`

4: `st r4,0(r3)`

5: `call spew_cash`

Processor 1

0: `addi r1,accts,r3`

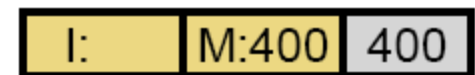
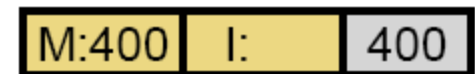
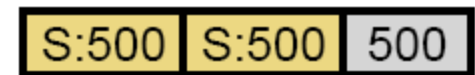
1: `ld 0(r3),r4`

2: `blt r4,r2,6`

3: `sub r4,r2,r4`

4: `st r4,0(r3)`

5: `call spew_cash`



- Υπόθεση: write-back caches (θα μπορούσε όμως και write-through...) + MSI protocol
- Τι συνέβη;
 - ...πάντως το *coherence protocol* λειτούργησε σωστά

Η ανάγκη για συγχρονισμό (3)

Processor 0

0: `addi r1,accts,r3`

1: `ld 0(r3),r4`

2: `blt r4,r2,6`

3: `sub r4,r2,r4`

4: `st r4,0(r3)`

5: `call spew_cash`

Processor 1

0: `addi r1,accts,r3`

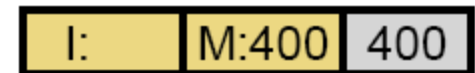
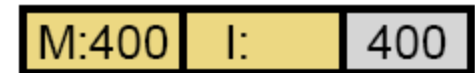
1: `ld 0(r3),r4`

2: `blt r4,r2,6`

3: `sub r4,r2,r4`

4: `st r4,0(r3)`

5: `call spew_cash`

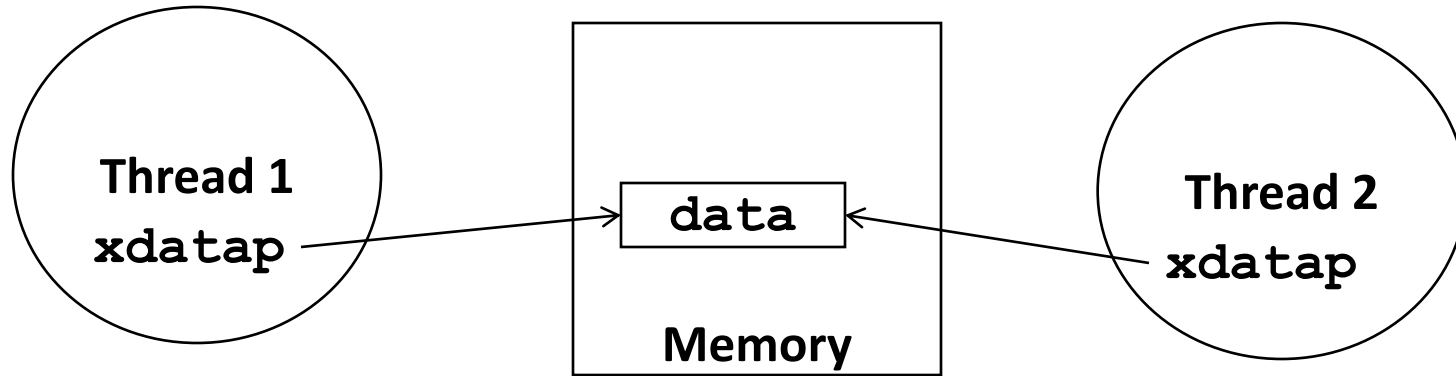


- Τι συνέβη στην πραγματικότητα;
 - διαισθητικά, η πρόσβαση (=read + modify + write) στον `accts[241].bal` θα πρέπει να γίνεται **ατομικά**
 - οι δοσοληψίες δε θα πρέπει να επικαλύπτονται μεταξύ τους
 - ...όμως αυτό ακριβώς έγινε!
 - λύση: **συγχρονισμός** προσβάσεων στον `accts[241].bal`

Συγχρονισμός

- Ρυθμίζει την πρόσβαση σε μοιραζόμενα δεδομένα
- **Κρίσιμη περιοχή** (critical section): όλες οι λειτουργίες εντός αυτής θα πρέπει να γίνονται *ατομικά*, σαν μία ενιαία και αδιαίρετη λειτουργία
- Πώς;
 - αμοιβαίος αποκλεισμός
 - locks, semaphores, monitors
 - atomic instructions
 - non-blocking μηχανισμοί

Παράδειγμα : Αμοιβαίος Αποκλεισμός



```
ld  xdata, (xdatap)
add xdata, 1
sd  xdata, (xdatap)
```

Τι χρειάζεται για να εκτελεστεί σωστά ο κώδικας;

Αμοιβαίος Αποκλεισμός με Load/Store (1)

- Χρήση 2 διαμοιραζόμενων μεταβλητών.

```
c1 = 1;  
L: if c2 == 1 then go to L;  
   <critical section>  
   c1 = 0;
```

```
c2 = 1;  
L: if c1 == 1 then go to L;  
   <critical section>  
   c2 = 0;
```

Πρόβλημα → **Deadlock!**

;

Αμοβαίος Αποκλεισμός με Load/Store (2)

- Χρήση 2 διαμοιραζόμενων μεταβλητών.

```
L: c1 = 1;  
  if c2 == 1 {  
    c1 = 0;  
    go to L;  
  }  
  <critical section>  
  c1 = 0;
```

```
L: c2 = 1;  
  if c1 == 1 {  
    c2 = 0;  
    go to L;  
  }  
  <critical section>  
  c2 = 0;
```

- Αποφυγή deadlock.
- Μικρή πιθανότητα *livelock*.
- Πιθανό *starvation*!

Αμοιβαίος Αποκλεισμός: Peterson (1981)

- Τροποποίηση του αλγορίθμου του Dekker (1966)
- Χρήση 3 διαμοιραζόμενων μεταβλητών.

```
c1 = 1;  
turn = 1;  
L: if c2 == 1 & turn == 1  
    go to L;  
    <critical section>  
    c1 = 0;
```

```
c2 = 1;  
turn = 2;  
L: if c1 == 1 & turn == 2  
    go to L;  
    <critical section>  
    c2 = 0;
```

- $turn = i \rightarrow$ μόνο το thread i μπορεί να περιμένει
- Οι $c1, c2$ εγγυόνται το mutual exclusion
- Πολύπλοκη λύση για n (Dijkstra)

Peterson (1981) vs Dekker (1966)

Peterson's:

"I want to enter."	flag[0]=true;
"You can enter next."	turn=1;
"If you want to enter and it's your turn I'll wait."	while(flag[1]==true&&turn==1){
Else: Enter CS!	}
"I don't want to enter any more."	// CS
	flag[0]=false;

Dekker's:

"I want to enter."	flag[0]=true;
"If you want to enter and if it's your turn I don't want to enter any more."	while(flag[1]==true){
"If it's your turn I'll wait."	if(turn!=0){
"I want to enter."	flag[0]=false;
	while(turn!=0){
	}
	flag[0]=true;
	}
	}
Enter CS!	// CS
"You can enter next."	turn=1;
"I don't want to enter any more."	flag[0]=false;

Αμοιβαίος αποκλεισμός

- Επιτρέπει την είσοδο στην κρίσιμη περιοχή μίας διεργασίας κάθε φορά
- Locks
 - acquire(lock) , release(lock)
 - δεύτερη προσπάθεια για acquire (από άλλη ή και την ίδια διεργασία) ενώ δεν έχει γίνει release, θα μπλοκάρει την εκτέλεση

```
struct acct_t { int bal; };  
shared struct acct_t  accts[MAX_ACCT];  
shared int lock;  
int id,amt;  
acquire(lock);  
if (accts[id].bal >= amt) {  
    accts[id].bal -= amt;           // critical section  
    spew cash(); }  
release(lock);
```

- Ζητούμενο: το acquire να γίνεται κι αυτό ατομικά!

Αμοιβαίος αποκλεισμός : ISA support

- Μπορεί να υλοποιηθεί με χρήση Load & Stores σε SC σύστημα ή και με την προσθήκη fences σε relaxed memory model.
 - Πολύπλοκος κώδικας
 - Μη αποδοτικός κώδικας
- Επέκταση ISA με ατομικές εντολές read-modify-write
 - Test and Set
 - Swap
 - Compare and Swap
 - Fetch and Increment, ...
 - Fetch-and- Φ : $\{t = \text{read}(x); \text{newval} = \Phi(t); \text{write}(\text{newval}, x)\}$

Spin-lock (“Test-And-Set”)

- Πολλές αρχιτεκτονικές παρέχουν εντολές για atomic lock acquisition

- Παράδειγμα: **test-and-set**

- **t&s r1, 0(&lock)**

- Εκτελεί ατομικά:

```
ld r1, 0(&lock)
st #1, 0(&lock)
```

- Αν το lock ήταν ελεύθερο (=0), το δεσμεύει (το θέτει σε 1)
- Αν το lock ήταν δεσμευμένο (=1), δεν το αλλάζει

- Acquire sequence:

```
A0: t&s r1, 0(&lock)
A1: bnez r1, A0
```

- Παρόμοιες εντολές: swap, compare & swap, exchange, fetch-and-add

Ορθότητα “Test-and-Set” Lock

Processor 0

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

CRITICAL_SECTION

Processor 1

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

- Ο P1 συνεχίζει να κάνει spin πάνω στο lock...

Απόδοση “Test-and-Set” Lock

Processor 1

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

Processor 2

A0: t&s r1,0(&lock)

A1: bnez r1,#A0

A0: t&s r1,0(&lock)

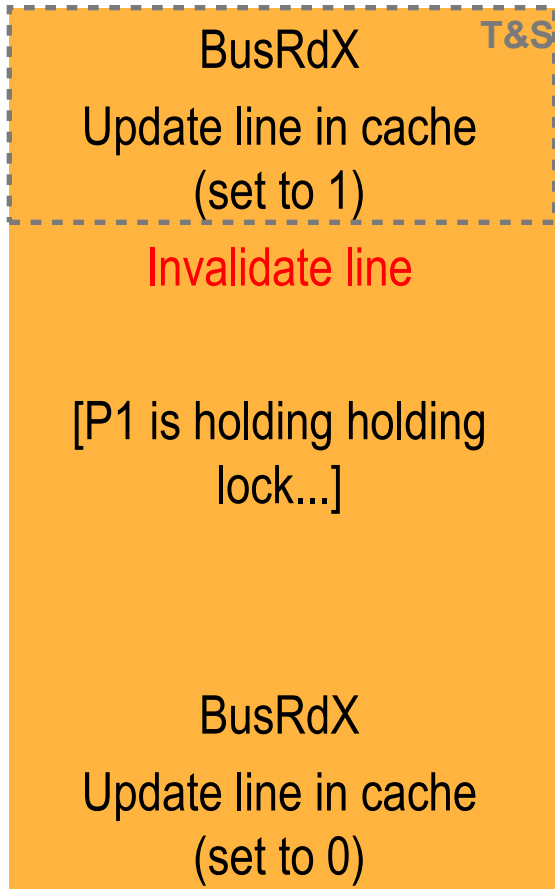
A1: bnez r1,#A0

M:1	I:	1
I:	M:1	1
M:1	I:	1
I:	M:1	1
M:1	I:	1

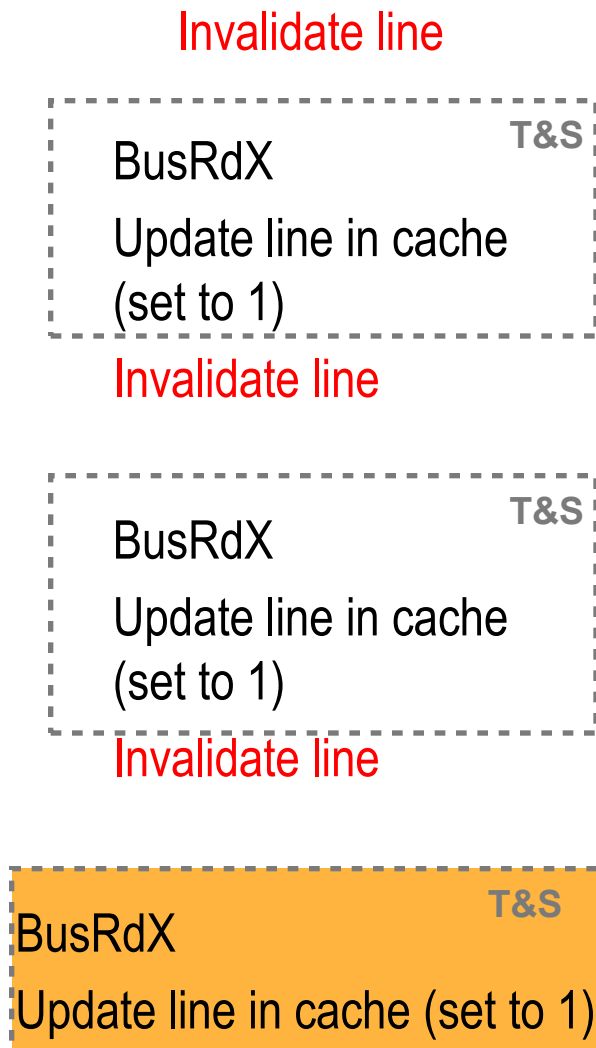
- ...επηρεάζοντας αρνητικά όμως την απόδοση
 - έστω ότι έχουμε 3 επεξεργαστές αντί για 2
 - ο P0 έχει το lock και είναι εντός της κρίσιμης περιοχής
 - τι κάνουν οι P1, P2 στο ενδιάμεσο;
 - εκτέλεση πολλών επαναλήψεων του t&s loop, κάθε μία εκ των οποίων περιλαμβάνει και ένα store
 - ο ένας ακυρώνει συνεχώς την cache line του άλλου, παράγοντας ιδιαίτερα αυξημένη (και μη χρήσιμη) κίνηση στο δίαυλο

Test&Set lock: consider coherence traffic

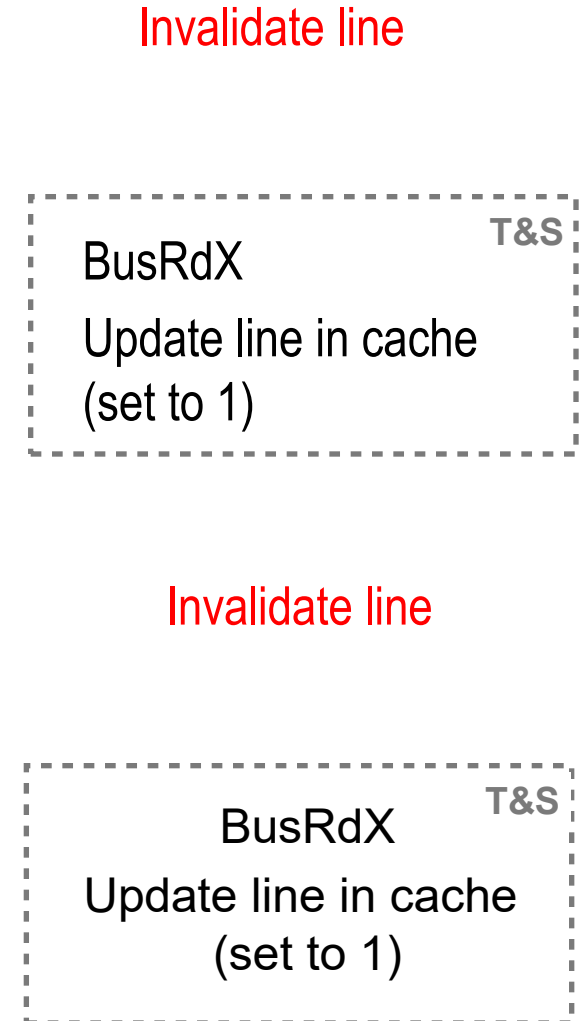
Processor 1



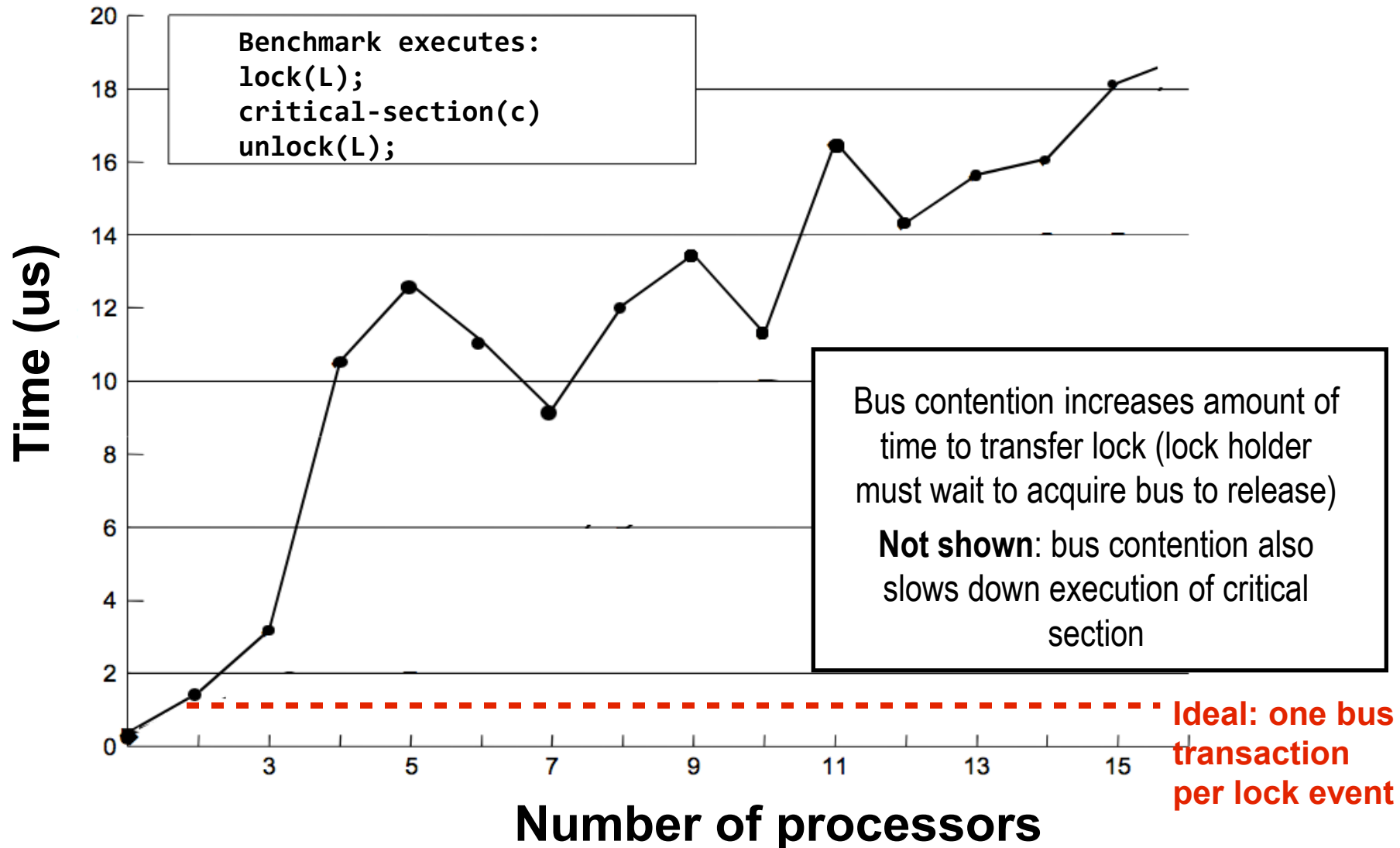
Processor 2



Processor



Test&Set lock performance



Test&Set lock with (exponential) back-off

Upon failure to acquire lock, delay for awhile before retrying

```
void Lock(volatile int* l) {  
    int amount = 1;  
    while (1) {  
        if (Test&Set(*l) == 0)  
            return;  
        delay(amount);  
        amount *= 2;  
    }  
}
```

“Test-and-Test-and-Set” Locks

- Σκεπτικό: αντί μία διεργασία να γράφει «τυφλά» στο lock μέσω t&s, να παρακολουθεί απλά την τιμή του και μόνο όταν φαίνεται να είναι ελεύθερο να επιχειρεί να το δεσμεύσει
- Acquire sequence:
A0: ld r1,0(&lock)
A1: bnez r1,A0
A2: addi r1,1,r1
A3: t&s r1,0(&lock)
A4: bnez r1,A0
- Μέσα σε κάθε επανάληψη του loop, πριν γίνει ένα t&s:
 - επαναληπτικά ελέγχουμε (load) να δούμε αν η τιμή του lock έχει αλλάξει
 - εκτελούμε το t&s (store) όταν το lock (φαίνεται να) είναι ελεύθερο
- Οι επεξεργαστές κάνουν spinning τοπικά στην cache τους
- Λιγότερη άχρηστη κίνηση στο δίαυλο

Απόδοση “Test-and-Test-and-Set” Lock

<u>Processor 1</u>	<u>Processor 2</u>
A0: ld r1,0(&lock)	
A1: bnez r1,A0	A0: ld r1,0(&lock)
A0: ld r1,0(&lock)	A1: bnez r1,A0
// lock released by processor 0	
A0: ld r1,0(&lock)	A1: bnez r1,A0
A1: bnez r1,A0	A0: ld r1,0(&lock)
A2: addi r1,1,r1	A1: bnez r1,A0
A3: t&s r1,(&lock)	A2: addi r1,1,r1
A4: bnez r1,A0	A3: t&s r1,(&lock)
CRITICAL_SECTION	A4: bnez r1,A0
	A0: ld r1,0(&lock)
	A1: bnez r1,A0

S:1	I:	1
S:1	S:1	1
S:1	S:1	1
I:	I:	0
S:0	I:	0
S:0	S:0	0
S:0	S:0	0
M:1	I:	1
I:	M:1	1
I:	M:1	1
I:	M:1	1
I:	M:1	1

- Ο P0 κάνει release to lock, και κάνει invalidate την αντίστοιχη cache line στους P1, P2
- Οι P1, P2 ανταγωνίζονται για την απόκτηση του lock, ο P1 κερδίζει

Atomic Instructions

- Πολλές αρχιτεκτονικές παρέχουν τη δυνατότητα ατομικής εκτέλεσης για συγκεκριμένες εντολές
 - όλες οι επιμέρους λειτουργίες που κάθε τέτοια εντολή περιλαμβάνει εκτελούνται σαν ένα ενιαίο σύνολο
 - συνήθως πρόκειται για Read-Modify-Write λειτουργίες
- π.χ. x86
 - INC, DEC, NOT, ADD, SUB, AND, OR, XOR,... (με LOCK prefix)
 - XCHG, CMPXCHG,...

```
int cmpxchg(int *p, int v1, int v2)
{ //atomically
    int oldval = *p;
    if (oldval == v1) *p = v2;
    return oldval;
}
```

- στα παλιότερα μοντέλα υλοποιούνται με κεντρικό κλείδωμα του bus (#LOCK signal)
 - στα νεότερα μοντέλα, κλειδώνονται μόνο οι caches που περιέχουν τα αντίστοιχα δεδομένα (αν τα περιέχουν)
- (+) Γενικά, πολύ πιο αποδοτικές σε σχέση με τα locks
- (-) Κατάλληλες για απλές λειτουργίες (π.χ. RMW), αλλά όχι για πιο σύνθετες

Barriers

Does this even work?

```
BARRIER(bar_name, p) {  
    LOCK(bar_name.lock);  
    if (bar_name.counter == 0)  
        bar_name.flag = 0;  
    bar_name.counter++;  
    UNLOCK(bar_name.lock);  
    if (bar_name.counter == p) {  
        bar_name.counter = 0;  
        bar_name.flag = 1;  
    }  
    else  
        while(bar_name.flag == 0) {}; /* busy wait */  
}
```

Barrier with sense reversal

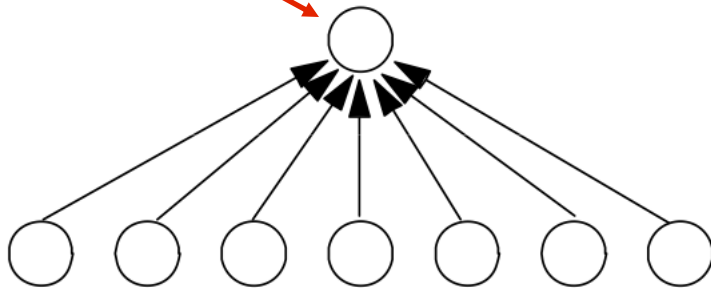
```
BARRIER(bar_name, p) {  
    local_sense = !(local_sense);    /* toggle private state */  
    LOCK(bar_name.lock);  
    bar_name.counter++;  
    UNLOCK(bar_name.lock);  
    if (bar_name.counter == p) {  
        bar_name.counter = 0;  
        bar_name.flag = local_sense;  
    }  
    else  
        while(bar_name.flag != local_sense) {}; /* busy wait */  
}
```

Centralized barrier: traffic

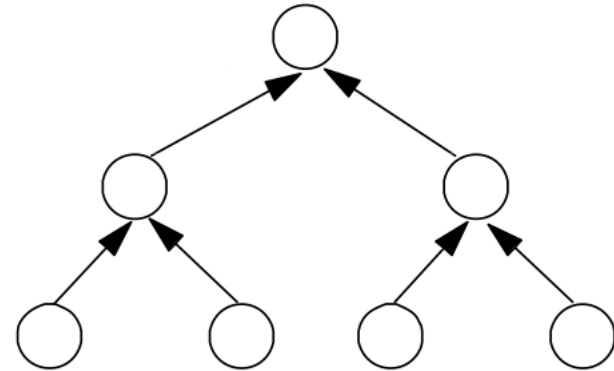
- ◆ $O(P)$ traffic on a bus:
 - $2P$ write transactions to obtain barrier lock and update counter
 - 2 write transactions to write flag + reset counter
 - $P-1$ transactions to read updated flag
- ◆ But there is still serialization on a single shared variable
 - Latency is $O(P)$
 - Can we do better?

Combining trees

High contention!



Centralized
Barrier

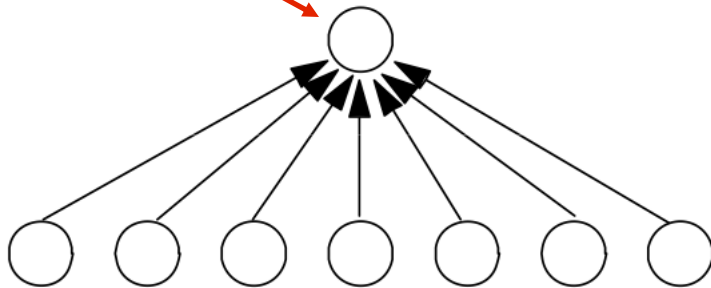


Combining Tree
Barrier

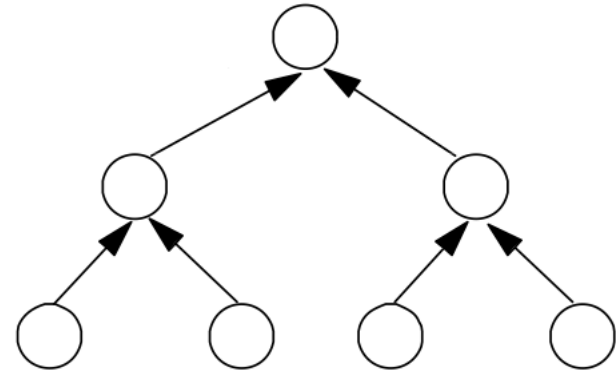
- ◆ Combining trees make better use of parallelism in interconnect topologies
 - $\log(P)$ latency
 - Strategy makes less sense on a bus (all traffic still serialized on single shared bus)
- ◆ Makes more sense when richer network (on- or off-

Combining trees

High contention!



**Centralized
Barrier**



**Combining Tree
Barrier**

- ◆ Acquire: when processor arrives at barrier, performs `atomicIncr()` of parent counter
 - Process recurses to root
- ◆ Release: beginning from root, notify children of release

Summary

- Locks
 - Trade-off between acquire speed (w/ or w/o contention), fairness
 - Test&Set, Test&Test&Set, with back-off
 - Array-based, Queue-based
- Other forms
 - Point-to-point synchronization
 - Barriers